

维度 2

基本算法与 数据结构模板

中文版 · 系统复习与模型识别

Senior C++ / 低延迟交易 / OA / Live Coding

*** 基础 / 必须掌握	** 需要掌握	* 可以了解 / 高阶扩展
能识别、解释不变量，并独立实现核心模板。	理解条件与证明，能写出主体或现场推导。	知道用途与限制，按岗位和题型深入。

从约束出发，识别结构；从不变量出发，选择模板。

16 个章节串起算法思维、数据结构、题型模型与设计范式。新增八个核心思想及双向索引，把具体模板与问题求解机制连接起来。

整理依据：所引用对话的可读取内容与本次补充要求。
修订版：2026-09-27 · 加入算法思维 · 适合 A4 打印 · 中文字体已嵌入

如何使用这本手册

星级表示本手册面向通用算法面试的复习优先级，不等同于算法难度，也不表示所有交易岗位都会考到。岗位明确要求某一领域时，应相应提升它的优先级。

使用情境	建议路线
首次系统复习	01 基础 → 02 算法思维 → 03 数据结构 → 04 核心算法 → 05 序列 → 06 树 → 07 图 → 08 设计范式；随后补齐其余章节。
陌生题无法下手	先看 02 章八个思想的提问表，再用 14 模型识别地图匹配候选算法；用前提与反例排除误用。
面试前快速回看	先看 14 模型识别地图、15 选型速查、16 实现检查表；每个知识卡的“思维关联”可点击回到第 02 章。
OA 训练	先估算数据规模与状态数，再选算法；优先复习 **，用边界输入和随机小样本检验。
Live Coding	先说输入契约与朴素解，再给不变量、代码和复杂度；完成后手动走查空输入、重复值和极值。
低延迟交易岗位	算法正确性之外，再看连续布局、分配次数、尾延迟、迭代器失效及并发模型；不要把面试模板直接当生产组件。

统一记号与代码约定 ***

n 为元素数， V/E 为图的点/边数， h 为树高， K 为窗口或选取规模， Q 为查询数， U 为值域， W 为机器字位数。复杂度默认说明额外空间；输出结果本身占用的空间另计。

数组与字符串优先使用 0-based 和半开区间 $[l,r)$ 。Fenwick 内部使用 1-based，外部仍是 0-based；图节点默认编号 $0..V-1$ 。树中父节点为 -1 或根自身的约定会明确说明。

代码以 C++17 为主体；`std::midpoint`、`std::popcount`、`std::bit_cceil` 等明确标注 C++20。模板为可组合函数/结构，省略标准头文件和 `std::` 前缀；需相应包含 `<algorithm>`、`<vector>`、`<queue>`、`<deque>`、`<numeric>`、`<limits>`、`<unordered_map>` 等，并使用 `std::` 限定名。

阅读边界：原对话的两条长回复在读取接口中被截断。本版覆盖可读取主题及本次列出的全部重点，并补齐常见基础内容；不宣称逐字收录截断部分。

从会背到会用 ***

对每个 *** 算法条目：给适用例子与失效反例，说明不变量，并独立实现和测试。对第 02 章的思维条目：能将同一思想解释到两种不同算法上，并说明转换、删除或合并的依据。 ** 重在理解推导， * 按需深入。

目录

目录和书签均可点击。章标题提供体系入口，二级条目可直接定位具体模板。

01 基础语言与正确性	7
1.01 复杂度与约束 ***	7
1.02 区间、索引与边界 ***	7
1.03 数值安全与未定义行为 ***	7
1.04 不变量、证明与反例 ***	7
02 算法思维与问题求解机制	8
2.01 三层框架与八个提问 ***	8
2.02 T1 改变问题的表达：转换与建模 ***	9
2.03 T2 排除不必要的候选：消除、剪枝与支配 ***	9
2.04 T3 合并等价情况：状态抽象与状态压缩 ***	10
2.05 T4 复用已经完成的工作：缓存、预处理与增量维护 ***	10
2.06 T5 分解问题，再组合结果 ***	11
2.07 T6 只维护目标查询需要的信息：充分摘要 ***	11
2.08 T7 改变计数视角：贡献、抵消与守恒 ***	12
2.09 T8 改变处理顺序：排序、离线与延迟执行 ***	12
2.10 正确性工具：不变量贯穿八个思想 ***	13
2.11 双向索引：同一算法可以组合多个思想 ***	14
2.12 完整推导与面试使用方法 ***	15
03 核心数据结构	16
3.01 连续容器、链表与队列 ***	16
3.02 哈希表与有序容器 ***	16
3.03 堆与优先队列 ***	16
3.04 Trie 与二进制 Trie **	17
3.05 并查集 DSU ***	17
3.06 前缀、差分与区间聚合 ***	18
3.07 Fenwick Tree **	18

3.08 Segment Tree 与 Sparse Table **	18
04 核心算法	21
4.01 排序与计数 / 桶思维 ***	21
4.02 Quickselect、Top K 与 Partition ***	21
4.03 边界二分与精确查找 ***	22
4.04 Binary Search on Answer ***	22
4.05 结构化二分 **	23
05 线性序列题型	24
5.01 双指针的五种角色 ***	24
5.02 滑动窗口与 exactly K ***	24
5.03 Prefix State: 统一前缀模型 ***	25
5.04 区间合并、交集与调度 ***	26
5.05 扫描线、事件与差分映射 ***	26
5.06 单调栈与贡献法 ***	26
5.07 单调队列 ***	27
5.08 Heap Patterns: Top K 与 K-way Merge ***	27
5.09 双堆中位数与惰性删除 **	27
5.10 链表反转、Dummy 与快慢指针 ***	28
06 树算法	29
6.01 遍历、递归返回值与 BST ***	29
6.02 LCA 与 Binary Lifting **	29
6.03 Euler Tour / Tree Flattening **	29
6.04 Tree DP、直径与换根 **	30
07 图算法	31
7.01 表示、DFS / BFS 与隐式图 ***	31
7.02 多源 BFS ***	31
7.03 拓扑排序、三色 DFS 与 DAG ***	32
7.04 Dijkstra ***	33

7.05 0-1 BFS、Bellman-Ford 与 Floyd **	33
7.06 MST: Kruskal 与 Prim **	34
7.07 SCC、桥、割点与双连通 **	34
7.08 二分图与状态染色 **	34
7.09 Functional Graph: 每点一个后继 **	34
08 算法设计范式	35
8.01 分治与归并计数 ***	35
8.02 贪心与证明 ***	35
8.03 回溯、去重与状态恢复 ***	36
8.04 DP 六问与状态表示 ***	36
8.05 线性、网格与状态机 DP ***	36
8.06 0/1、完全与多重背包 ***	37
8.07 LIS: DP 与 tails ***	37
8.08 字符串 DP: LCS、编辑距离与回文 ***	38
8.09 区间 DP 与状态压缩 DP **	38
09 特殊面试技巧	39
9.01 Bit 操作与无符号语义 ***	39
9.02 子集、子掩码与位迭代 **	39
9.03 Missing / Duplicate 决策树 ***	40
9.04 In-place Indexing 与原地压缩 ***	42
9.05 多数投票与候选消除 ***	43
9.06 Prefix / Suffix、排列与压缩 ***	43
9.07 Offline Processing: 重排查询 **	44
9.08 DSU 扩展: 元数据、动态点与势能 **	44
9.09 随机化与抽样 **	44
10 字符串算法	45
10.01 扫描、频数与字符边界 ***	45
10.02 KMP 与 Z Algorithm **	45
10.03 Rolling Hash / Rabin-Karp **	46

10.04 回文：中心扩展与 Manacher **	46
10.05 多模式与后缀结构 *	47
11 数学与数论	48
11.01 GCD、LCM 与抽屉原理 ***	48
11.02 快速幂与模运算 ***	48
11.03 筛法与质因数分解 **	49
11.04 组合计数与模逆 **	49
11.05 扩展数论与矩阵递推 *	49
11.06 计算几何基础 *	49
12 搜索空间优化	50
12.01 剪枝、记忆化与状态编码 ***	50
12.02 Dominance / Pareto 状态剪枝 **	50
12.03 Bidirectional BFS **	50
12.04 Meet-in-the-middle **	51
12.05 迭代加深、Branch and Bound 与 A* *	51
13 高级结构、算法与岗位扩展	52
13.01 Lazy、持久化与树链剖分 *	52
13.02 平衡树、Treap 与 Order Statistics *	52
13.03 离线高级：Mo、CDQ、回滚 DSU *	52
13.04 网络流、最小割与匹配 *	53
13.05 Digit DP、DP 优化与 XOR Basis *	53
13.06 LRU / LFU、Bloom Filter 与一致性哈希 **	53
13.07 低延迟数据布局与并发边界 **	54
14 模型识别地图	55
15 复杂度与算法选型速查	58
16 C++ 面试实现检查表	60

01 基础语言与正确性

统一复杂度、索引、数值和证明方式，为后面的模板提供共同契约。

1.01 复杂度与约束 ***

思维关联 共同基础：衡量候选、状态与计算量，以及时间和空间的交换

解决什么 / 识别信号 判断方案能否在时间和内存限制内完成；先读 n 、 Q 、值域、图密度和时间上限

核心思想 区分最坏、平均、期望与摊还；总成本是各阶段之和，嵌套循环不一定相乘，要数实际推进次数

复杂度 / 不变量 单调双指针 $O(n)$ ；哈希平均 $O(1)$ 、单次最坏 $O(n)$ ；vector 尾插摊还 $O(1)$ 。不变量：每个对象被计费的次数有上界

常见坑 / 替代方案 把摊还误当成每次固定耗时；忽略输出规模、递归栈和哈希常数。关联：第 15 章规模表；低延迟场景还看单次最坏路径与尾延迟

1.02 区间、索引与边界 ***

思维关联 共同契约：所有变换、分解和不变量都依赖一致的边界语义

解决什么 / 识别信号 减少越界和差一错误；子数组、二分、分治、区间查询

核心思想 统一 $[l, r)$ ：长度 $r-l$ ，空区间 $l=r$ ；最后元素 $r-1$ 。题目闭区间 $[L, R]$ 可转为 $[L, R+1)$ ，但先检查 $R+1$ 是否可表示

复杂度 / 不变量 约定本身无额外渐进成本。不变量： $0 \leq l \leq r \leq n$ ，循环中持续成立

常见坑 / 替代方案 混用 0/1-based； $\text{size}()-1$ 在空容器上无符号下溢。关联：闭区间模板也可用，但必须全程一致

1.03 数值安全与未定义行为 ***

思维关联 共同契约：机器运算必须实现证明中使用的数学关系

解决什么 / 识别信号 保证数学推导与机器计算一致；求和、乘积、距离、位移、比较器

核心思想 先估算最大中间结果。乘法先扩宽；除法前检查除数；有符号溢出不是可靠的回绕。INF 留余量并保护加法

复杂度 / 不变量 机器整数通常 $O(1)$ ；大整数成本依位数增长。不变量：每个中间值均在所选类型与哨兵范围内

常见坑 / 替代方案 $1LL*a*b$ 仍可能超 long long； $\text{abs}(\text{INT_MIN})$ 不可表示；比较器不要用 $a-b$ 判符号。关联：约束内使用 long long；更宽整数须说明编译器支持；需要时用受检算术

1.04 不变量、证明与反例 ***

思维关联 正确性工具层：初始化、保持、终止与反例检验

解决什么 / 识别信号 解释模板为什么正确；面试官追问正确性；边界分支难以确认

核心思想 依次说明初始化、保持、终止。递归用归纳，贪心用交换论证或领先论证；反例优先找重复值、负数和单调情形

复杂度 / 不变量 证明不增加运行时间；会影响选型。不变量：当前维护的含义与代码每一步一致

常见坑 / 替代方案 仅凭几组样例认为正确；把经验规律当单调性证明。关联：第 08 章范式；第 16 章测试检查表

数值例子：int a,b; long long x = 1LL*a*b; 只解决 int 乘法阶段的溢出。二分 $\text{lo} + (\text{hi} - \text{lo}) / 2$ 要求 $\text{hi} - \text{lo}$ 可表示；一般整数端点可用 $\text{C++20 std::midpoint}$ 。取模归一化可写 $r = x \% k$; if ($r < 0$) $r += k$, 其中 $k > 0$ 。

02 算法思维与问题求解机制

在基础契约与具体模板之间，建立“怎样发现解法”的思维入口。本章围绕候选解、状态和计算工作展开，并与后续知识卡双向连接。

2.01 三层框架与八个提问 ***

面对陌生题，可以依次问：怎样表达问题才能暴露结构？怎样减少必须考虑的候选、状态或计算？为什么减少之后仍能得到正确答案？这三问分别对应问题表达、求解机制与正确性依据。

层次	核心任务	常见工具
问题表达	明确对象、目标、约束与需要保留的关系	建模、转换、状态定义
求解机制	减少候选、不同状态、重复工作或维护信息	排除、合并、复用、分解、摘要、计数、重排
正确性依据	说明变换、删除、合并与重排为什么安全	不变量、单调性、等价性、交换论证、归纳

八个核心思想：陌生题的提问表 ***

编号 / 思想	可以直接问自己的问题	典型落点
T1 转换与建模	换一个对象、表示或目标，会不会更容易？	子树→区间；优化→判定
T2 排除候选	哪些候选不可能成功，或不可能更优？	二分、剪枝、支配、贪心
T3 合并等价状态	不同历史是否拥有相同的未来选择？	DP、记忆化、对称去重
T4 复用计算	哪部分已经算过，或仅有局部变化？	前缀、窗口、KMP、倍增
T5 分解与组合	怎样切分，子问题之间还剩什么联系？	分治、树 DP、区间 DP、MITM
T6 充分摘要	为目标查询，需要保留哪些信息？	Top K、DSU、聚合、奇偶掩码
T7 贡献与抵消	能否换一个计数主体，或消去公共部分？	贡献法、XOR、差分、多数投票
T8 调整处理顺序	能否排序、逆序、按依赖或延迟处理？	离线、扫描线、拓扑、Lazy

使用规则：T1-T8 是可组合的观察角度，允许一题多标签。它们的星级 *** 表示思维训练优先级；映射到的具体算法仍按原知识卡标为 */**/***。基础契约与证明工具贯穿所有思想，不必强行归入某个算法类别。

2.02 T1 改变问题的表达：转换与建模 ***

减少什么 减少表达障碍，让难操作的对象变成已有结构能处理的对象。转换本身未必降低复杂度，但可能暴露新的解法。

何时尝试 问题包含网格、字符串、后继、子树、巨大值域，或“最小化最大值”等间接目标。

机制 / 正确性 先列出答案依赖的关系，再定义新对象与映射。可换对象、表示、目标或关系。证明原解怎样映射到新解，以及新答案怎样还原；若仅保留部分性质，必须明确其适用查询。

知识点归位 Euler Tour、Binary Search on Answer、Prefix State、隐式图、Functional Graph、Floyd Duplicate、Coordinate Compression、In-place Indexing、Trie、Bit 编码、网络流建模。

边界 / 反例 离散化保序而不保距离；优化转判定仍需证明判定单调与 check 正确；值域不符时不能把数组随意解释成后继图。

推演例子：树上子树求和：DFS 把同一子树连续访问，得到 [tin,tout)；按 tin 存值后使用区间结构。先证明“子树恰好对应这个区间”，再讨论 Fenwick 的更新与查询。

数组转后继图：Functional Graph 推演 **

next=[1,2,3,1,2]：1→2→3→1 是长度 3 的环；0 距环 1 步且入口为 1；4 距环 1 步且入口为 2。求 0 的第 k 个后继：先走到环，剩余步数对环长取模。多起点多查询可用 $up[v][j]=up[up[v][j-1]][j-1]$ 。从数组表示转成“入环路径+周期”，就暴露了可利用的结构；算法详见第 07 章。

2.03 T2 排除不必要的候选：消除、剪枝与支配 ***

减少什么 减少继续枚举的候选解、分支或状态。理由可以是不可行、不能改进、被支配，或与已考虑情形重复。

何时尝试 候选空间很大，但一次比较、约束检查或界的计算能排除一批情况；存在单调性、最优性界或可比较的资源状态。

机制 / 正确性 删除之前先给“不会丢失目标”的证明。寻找一个最优解时，只需保留至少一个最优候选；枚举所有解时则要保留全部目标，或保留足够的计数/重建信息。

知识点归位 Binary Search、Two Pointers、Quickselect、Monotonic Stack/Queue、Backtracking Pruning、Dominance/Pareto、Branch and Bound、部分贪心、LIS tails、凸包。

边界 / 反例 可行并不等于值得保留；支配也不等于等价。求最优值时可剪掉只能打平的分支，但枚举所有最优解时往往不能。负代价可能使“当前太贵”失去剪枝依据。

推演例子：窗口最大值里，旧候选值为 3，新候选值为 5。新元素不小且更晚过期，旧元素在未来窗口中不会更优，因此可永久删除。队列单调是这种支配删除的结果。

2.04 T3 合并等价情况：状态抽象与状态压缩 ***

减少什么 减少被区别对待的历史数量，把对未来影响相同的历史归为同一状态；位掩码本身只是编码方式，不自动减少状态数。

何时尝试 搜索反复到达相似局面；完整路径很长，但未来似乎只取决于位置、剩余资源、末点或已选集合。

机制 / 正确性 检查同一状态是否具有相同的后续动作与成本规则。随后按目标合并状态值：计数累加、最优值取 min/max、可行性取 OR。历史的累计代价可以不同，合并时仍须保留正确聚合值。

知识点归位 DP 状态设计、Memoization、Bitmask DP、状态图 visited、对称性去重、Digit DP、SCC 缩点、Suffix Automaton。

边界 / 反例 只记坐标会遗漏钥匙或预算；只记 mask 可能遗漏末点。SCC 缩点对分量可达性有效，不能据此删除内部距离信息。自动机状态等价也必须相对于指定的未来行为。

推演例子：0/1 背包的状态 (i, c) 表示仅考虑前 i 个物品、容量限制为 c 。未来物品相同，因此可用一个最优值代表该状态；若去掉 i 而仍允许任意物品选择，就可能错误重复使用物品。

2.05 T4 复用已经完成的工作：缓存、预处理与增量维护 ***

减少什么 减少相同计算和相邻问题间重叠的工作；收益来自共享结果，而代价通常是额外存储与维护。

何时尝试 很多相似查询、重叠子问题、逐步变化的窗口，或长操作可以由已算出的短操作组合。

机制 / 正确性 三种路径：相同子问题只算一次；提前计算公共信息；变化时只更新受影响部分。缓存键要包含全部相关状态，增量公式要覆盖所有变化，预处理结果要与当前数据版本一致。

知识点归位 Memoization、DP、Prefix、Fenwick、Segment Tree、Sparse Table、Sliding Window、Binary Lifting、KMP/Z、Manacher、Rolling Hash、持久化结构、筛法。

边界 / 反例 输入更新后旧缓存可能失效；KMP 复用的是已证明的匹配关系；哈希摘要存在碰撞；数据规模过大时预处理空间或常数可能超过收益。

推演例子：固定长度窗口从 $[l, r)$ 变为 $[l+1, r+1)$ ，窗口和只需减 $a[l]$ 、加 $a[r]$ 。滑动中位数仍需维护顺序结构，不能因为窗口只变两个元素就假设更新一定 $O(1)$ 。

2.06 T5 分解问题，再组合结果 ***

减少什么 减少一次需要共同处理的对象数量；只有分解后的子问题与合并成本足够低，总体复杂度才会改善。

何时尝试 自然存在左右区间、孩子子树、前后两侧、集合两半、路径分段，或最后一次操作可以作为切分点。

机制 / 正确性 写出分解边界、子问题返回信息、跨边界关系及合并规则。证明所有目标均被覆盖，且重复贡献得到正确处理；用递推式或状态数核算真实成本。

知识点归位 Divide and Conquer、Merge Sort、Merge Sort Counting、Tree DP、Interval DP、Meet-in-the-middle、Prefix/Suffix、分块、HLD、快速幂、矩阵幂。

边界 / 反例 把问题拆成两半后仍枚举全体笛卡尔积，可能没有改善；子问题并非独立时需把依赖纳入状态；输出全部排列的规模不会被分治消除。

推演例子：逆序对=左半内部+右半内部+跨左右边界。两边排好序后，若 $\text{left}[i] > \text{right}[j]$ ，当前 $\text{right}[j]$ 一次产生 $\text{mid}-i$ 个贡献。难点是跨边界计数，合并后又为上一层提供有序输入。

2.07 T6 只维护目标查询需要的信息：充分摘要 ***

减少什么 减少维护的信息细节或顺序要求。摘要是否充分，取决于目标查询和未来允许的更新。

何时尝试 只问连通性、区间聚合、前 K 个、频次奇偶、当前最值等有限性质；维护全量关系或完全有序可能过重。

机制 / 正确性 先写接口，再判断状态能否回答每种查询并支持每种更新。DSU 的连通分区可回答连通性，不能还原路径；堆保存全部堆元素，只弱化其顺序要求；Top K 堆才会丢弃其余元素。

知识点归位 Heap/Top K、DSU 及元数据、区间聚合、双堆、Trie 计数、Parity Mask、频数桶、Boyer-Moore、Bloom Filter、XOR Basis、带子树大小的有序树。

边界 / 反例 区分精确充分信息、候选生成和概率摘要。多数投票通常还需验证；哈希相等可能碰撞；Bloom Filter 有假阳性。双堆求精确中位数通常仍保存所有活动元素，摘要不意味着常数空间。

推演例子：维护流中前 K 大值，只需大小 K 的最小堆。新值 $\leq$ 堆顶且堆已满时可忽略；若后来要求“撤销任意历史元素”，这个被丢弃的数据策略就不再自动满足新接口。

2.08 T7 改变计数视角：贡献、抵消与守恒 ***

减少什么 减少逐答案枚举的工作，改为按元素、边界或公共部分组织统计；计数目标本身保持不变。

何时尝试 要求所有子数组/配对的总和或数量，或出现固定频率、前缀关系、区间统一加值等可抵消结构。

机制 / 正确性 贡献法给每个答案指定唯一负责者，或证明重计次数可被修正；抵消法先说明被消掉的部分及保留量。不变量可以是总和、奇偶、净余额或每个对象的归属。

知识点归位 Contribution Method、Prefix Sum/XOR、Difference、子数组最小值之和、逆序对计数、Missing/Single Number、Boyer-Moore、组合计数、前后缀分解。

边界 / 反例 重复值需要一致的严格/非严格边界，避免一个子数组由多个相同最小值认领；XOR 只保留位奇偶，不能恢复任意频数；抵消后的多数候选不等于已验证答案。

推演例子：求所有子数组最小值之和：每个 $a[i]$ 的贡献为 $a[i] \times \text{左侧可选端点数} \times \text{右侧可选端点数}$ 。对重复最小值规定一侧严格、一侧非严格，使每个子数组恰有一个负责者。

2.09 T8 改变处理顺序：排序、离线与延迟执行 ***

减少什么 减少重复维护或困难操作，把处理安排成单调推进、依赖先满足或按需执行的过程。

何时尝试 查询全部已知；数据可排序；只有删除但难以维护；状态有依赖；大量更新可以合并到真正需要时处理。

机制 / 正确性 重新排序暴露结构，反向处理把删除变添加，拓扑顺序保证依赖，Lazy 延迟分发。正确性要求顺序变更不会改变题意，或能通过保留 id、快照与标记恢复原语义。

知识点归位 Sorting+Scan、Sweep Line、Offline DSU、逆序删边、Topological Order、DP 循环顺序、Lazy Propagation/Deletion、Mo、CDQ、回滚 DSU、随机顺序与缓存淘汰。

边界 / 反例 交互式或依赖上一个答案的查询未必可离线；同坐标事件顺序影响端点语义；Lazy 中赋值与加法的组合顺序不能交换；调整执行顺序不等于任意重排业务事件。

推演例子：按阈值 T 查询连通性：查询按 T 排序，边按权值排序，逐步加入权 $\leq T$ 的边并用 DSU 回答，再按查询 id 恢复输出。若条件变成权 $< T$ ，等值边的加入时机也必须改变。

2.10 正确性工具：不变量贯穿八个思想 ***

不变量是每次到达某个程序位置时始终成立的事实。它不是单调性的同义词，也不直接等于算法正确性；还要证明初始化成立、每一步保持，并在终止时推出目标结论。数学证明最后还要落实到索引、整数范围和对象生命周期。

工具	要证明的内容	典型对应
不变量 ***	运行中一直保持的状态含义；初始化、保持、终止	二分候选域、窗口合法性、DSU 分区、堆性质
单调性 ***	条件或指标沿某个明确方向只变化一次或不反向	二分答案、端点推进、离线阈值激活
等价 / 保真 ***	变换对目标查询保留哪些解或关系	Euler Tour、状态合并、SCC 缩点
归纳 / 依赖 ***	较小子问题正确时，当前组合为何正确	递归、树 DP、区间 DP、拓扑 DP
交换 / 安全选择 ***	可以构造采用当前选择且不更差的最优解	区间调度、MST 割性质、部分贪心
界 / 支配 **	被删除分支不可改进；或其所有延续可被另一状态替代	Branch and Bound、Pareto、单调队列
摊还 / 概率 **	总工作为何受限，或结论为何是期望/概率性质	单调结构、DSU、随机选择、水塘抽样

把算法名称换成一条可检查的陈述 ***

算法	核心陈述	不能省略的条件
Binary Search	左侧已证 false，右侧已证 true，未知区间每轮变小	predicate 单调，端点与无解语义一致
Sliding Window	调整后窗口合法；计数时合法左端集合形状已证明	不是任意负数求和都能左右缩放
Monotonic Queue	候选未过期，且被删除者被更晚候选支配	值比较和过期条件均正确
DP	状态含义足够，转移枚举合法选择，读取的依赖已正确	压缩空间时不能混淆新旧层
DSU	根表示连通分量，成功合并两个不同根才减分量数	普通版本不能任意删边
Dijkstra	有效最小距离弹出时已能定案	所有边权非负；跳过过期状态
Euler Tour	一次进入序中子树恰好是连续区间	区分 n 长子树序与 2n-1 长完整游走

工程验证顺序：先验证数学模型，再检查机器实现，再用边界样例和小规模对拍寻找反例。测试能发现错误；通过若干测试本身不能代替对所有合法输入的证明。

2.11 双向索引：同一算法可以组合多个思想 ***

思想 → 算法：先读 T1-T8 的“知识点归位”，再用下表跳到对应章节。算法 → 思想：后续每个原有知识卡新增可点击的“思维关联”，返回本章。标签表示理解视角，主标签在前；不要求每个算法只属于一类。

知识点	主思想 + 辅助思想	正确性抓手	章节
Binary Search on Answer	T1 转判定 + T2 缩候选	可行性单调，check 自身正确	04
Prefix State	T1 转关系 + T6 摘要 + T7 抵消	前缀定义与消去规则	05
DP / Memoization	T3 状态 + T4 复用 + T5 组合	状态充分，转移与初值正确	08
Monotonic Queue	T2 支配 + T4 增量	更优且更晚过期	05
Euler Tour + Fenwick	T1 展平 + T4 复用 + T6 聚合	子树连续，范围聚合准确	06 / 03
Offline DSU	T8 重排 + T4 增量 + T6 分区	激活边恰好满足当前阈值	09 / 03
KMP / Z	T4 复用 + T2 排除	复用范围已有匹配证明	10
Merge Sort Counting	T5 分解 + T7 贡献 + T8 排序	内部分开计，跨边界不重不漏	08
LIS tails	T2 支配 + T6 摘要	同长度更小尾值不限制未来	08
Missing / Duplicate	T1 建模或 T7 抵消	值域与出现次数满足具体契约	09
Dijkstra / A*	T8 顺序 + T2 安全定案/界	非负边；A* 的启发式与重开规则	07 / 12
Lazy / Persistent	T8 延迟或 T4 共享	标记组合正确，旧版本不可变	13

辨析：DP、剪枝与贪心不是同一个动作 ***

DP 按未来所需信息定义状态，再正确聚合不同历史；剪枝利用不可行性、界或支配关系跳过分支；贪心则在证明安全后承诺当前选择。三者可组合，但不能把“看起来更好”直接当成状态等价、可剪枝或安全贪心的证据。

覆盖方式：原有 81 张知识卡均保留并增加思维关联。其中数值安全、索引契约等归入跨思想基础；概率算法与低延迟实现还需概率模型、成本模型和生命周期等专项依据，不能只靠 T1-T8 标签解释全部正确性。

2.12 完整推导与面试使用方法 ***

例一：带负数的连续子数组计数 ***

题目：统计和为 K 的子数组。先用 T1 把区间和改写成 $S[j]-S[i]=K$ ；用 T6 判断只需历史前缀值的频数；用 T4 逐步更新频数；用 T7 说明公共前缀被抵消。因为要求 $i<j$ ，所以先查 $S[j]-K$ ，再加入 $S[j]$ ；空前缀频数为 1。

反例检查：负数使常规“和大就缩左端”的单调性失效，因此不能直接套求和窗口。若改求最长长度，摘要应改成每个前缀值的最早位置；只换返回值而不改维护信息是不够的。对应正文：第 05 章 Prefix State。

例二：最小化最大段和 ***

题目：非负数组分为不超过 K 个连续段。T1 把优化转成“容量 cap 是否可行”；给定 cap 后，尽量装满当前段的贪心得到最少段数；T2 再利用可行性随 cap 增大不变差的性质排除答案范围。

这里有两段独立证明：check 的贪心为何最少分段，以及答案域为何单调。只有第二段正确仍不够。循环不变量再保证二分始终保留最优可行值。若允许负数，贪心与上下界都要重审。对应正文：第 04 章二分答案。

例三：动态子树统计 **

题目：树上单点更新与子树求和交错。T1 用 Euler Tour 把树映成数组区间；T5 用 Fenwick 的固定块拆分前缀；T4 复用块聚合；T6 每块仅维护求和所需的信息。复杂度为 $O(n)$ 展平，之后每次更新或查询 $O(\log n)$ 。

反例检查：把“子树”改成任意两点路径后，简单 $[tin,tout)$ 就不能表示目标；把“和”改成任意区间 min，普通前缀差也失效。模型和聚合操作都决定结构选择。对应正文：第 06 章 Euler Tour 与第 03 章范围结构。

把思路写成可审核的六句话 ***

顺序	填写内容
1 / 契约	对象、目标、输入范围、可修改性与在线要求是什么？
2 / 表达	选择什么状态或变换？保留了哪些与答案有关的关系？
3 / 减少	准备排除哪些候选、合并哪些历史、复用哪些计算？
4 / 证明	变换为何安全？不变量是什么？哪里依赖单调性或交换论证？
5 / 成本	状态数、转移数、维护次数、额外空间分别是多少？
6 / 反例	改变重复值、符号、边权或查询顺序后，哪一项前提会失效？

练习标准：同一个思想找两个不同领域的例子；同一个算法用至少两个思想解释；再构造一个破坏关键前提的反例。目标是能推导模板，并知道何时应停止套用它。

03 核心数据结构

本章回答“存什么、支持哪些操作”。应用模型集中在后续章节，避免把结构与题型混为一层。

3.01 连续容器、链表与队列 ***

思维关联 T1 转换与建模 · T8 调整处理顺序

布局与访问顺序共同决定实现成本

解决什么 / 识别信号 在访问、插删、稳定性之间取舍；按下标访问、双端操作、节点搬移

核心思想 vector 连续布局；deque 分段存储且支持双端；list 通过节点链接。stack/queue 是接口适配器，不等于独立内存结构

复杂度 / 不变量 vector 访问 $O(1)$ 、中间插删 $O(n)$ ；deque 双端 $O(1)$ ；list 已知迭代器插删 $O(1)$ 、查找 $O(n)$ 。不变量：容器大小、有效区间和迭代器始终一致

常见坑 / 替代方案 把“链表插入 $O(1)$ ”误用到还需要 $O(n)$ 查位置的情况；忽略缓存局部性。关联：多数面试优先 vector；队列可用 queue/deque 或 vector+head

3.02 哈希表与有序容器 ***

思维关联 T6 充分摘要 · T8 调整处理顺序

等值索引或有序索引保留不同的查询能力

解决什么 / 识别信号 去重、频数、映射、前驱后继；只问等值用 hash；需要顺序/范围用 map 或 set

核心思想 unordered_map/set 依哈希桶；map/set/multiset 支持有序定位。提前 reserve 可减少 rehash，不能消除冲突

复杂度 / 不变量 哈希查改平均 $O(1)$ 、最坏 $O(n)$ ；有序容器 $O(\log n)$ ；空间 $O(n)$ 。不变量：相等键必须具有相等哈希；比较器必须严格弱序

常见坑 / 替代方案 operator[] 会插入；unordered rehash 使迭代器失效；multiset.erase(value) 删除全部等值元素。关联：稠密小值域用 vector 频数；静态数据可排序后二分；自定义 hash **、flat hash *

3.03 堆与优先队列 ***

思维关联 T6 充分摘要 · T8 调整处理顺序

维护局部顺序，并按当前最优候选推进

解决什么 / 识别信号 反复取当前最优候选；数据流、Top K、多路合并、调度

核心思想 堆只保证父子顺序；priority_queue 默认最大堆，greater<T> 得最小堆。排序不变量比全局有序弱

复杂度 / 不变量 建堆 $O(n)$ ；push/pop $O(\log n)$ ，top $O(1)$ ，空间 $O(n)$ 。不变量：堆顶为当前全体候选中的最值

常见坑 / 替代方案 把堆当成可随机删除或整体有序容器；比较器方向写反。关联：需要任意删除可用 multiset，或版本号+惰性删除；第 05 章堆题族

3.04 Trie 与二进制 Trie **

思维关联 T1 转换与建模 · T4 复用计算

共享前缀路径；二进制表示暴露逐位选择

解决什么 / 识别信号 共享前缀检索和最大 XOR；大量前缀查询、词典、按位贪心

核心思想 字符 Trie 每条边消耗一个字符；二进制 Trie 每层消耗一位，求最大 XOR 优先走相反位

复杂度 / 不变量 字符操作 $O(L)$ ，节点至多总字符数+1；W 位 Trie 每次 $O(W)$ ，空间 $O(nW)$ 。不变量：节点对应已消费的前缀；计数代表仍存在的词/值

常见坑 / 替代方案 固定字母表数组占用大；删除需维护计数；UTF-8 字节不等于字符。关联：单模式匹配 KMP；静态词集可排序；多模式 AC 见第 10 章

3.05 并查集 DSU ***

思维关联 T6 充分摘要 · T4 复用计算

维护连通分区，并复用压缩后的根路径

解决什么 / 识别信号 维护不断合并的无向连通分量；连通判断、账户合并、Kruskal、加边

核心思想 根代表集合；路径压缩加按大小合并。只有两个不同根合并时，分量数减一

复杂度 / 不变量 m 次操作总计 $O((n+m)\alpha(n))$ ，空间 $O(n)$ 。不变量：每个点到根表示集合归属，size 只在根处有效

常见坑 / 替代方案 普通 DSU 不支持任意删边，不解决有向可达；不要直接连非根节点。关联：静态连通分量用 DFS/BFS；扩展 DSU 与离线删除见第 08、12 章

C++ 模板 · DSU

节点编号必须位于 $[0, n)$ 。该版本用于普通合并，不用于回滚。

```
struct DSU {
    vector<int> p, sz;
    int components;
    explicit DSU(int n) : p(n), sz(n, 1), components(n) {
        iota(p.begin(), p.end(), 0);
    }
    int find(int x) {
        while (x != p[x]) {
            p[x] = p[p[x]];
            x = p[x];
        }
        return x;
    }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a; sz[a] += sz[b]; --components;
        return true;
    }
};
```

3.06 前缀、差分与区间聚合 ***

思维关联 T4 复用计算 · T7 贡献与抵消

共享前缀或通过边界变化累计区间贡献

解决什么 / 识别信号 静态范围查询或批量范围加值；先建一次查很多次；所有更新最后统一输出

核心思想 $\text{prefix}[r] - \text{prefix}[l]$ 求 $[l, r)$ 和；差分 $d[l] += x, d[r] -= x$ ，最后前缀还原。二维前缀用容斥

复杂度 / 不变量 前缀构建 $O(n)$ 、查询 $O(1)$ ；差分每次更新 $O(1)$ 、恢复 $O(n)$ ；空间 $O(n)$ 。不变量： $\text{prefix}[i]$ 表示前 i 项；差分是相邻前缀恢复值的变化量

常见坑 / 替代方案 差分不是在线范围和结构；二维角点符号错；溢出。关联：Fenwick/Segment Tree 支持交错更新查询；Prefix State 见第 05 章

3.07 Fenwick Tree **

思维关联 T4 复用计算 · T6 充分摘要

复用固定块聚合，避免重复扫描区间

解决什么 / 识别信号 动态单点增量与前缀和；排名计数、逆序对、频次、第 k 个

核心思想 内部索引 i 的节点覆盖以 i 结尾、长度 $\text{lowbit}(i)$ 的区间；更新向上，查询向下

复杂度 / 不变量 构建用逐点加 $O(n \log n)$ ，可线性建树；更新/查询 $O(\log n)$ ，空间 $O(n)$ 。不变量： $\text{bit}[i]$ 聚合其负责的固定区间

常见坑 / 替代方案 内部索引 0 会死循环；set 操作需转 delta ；范围和依赖可逆的前缀差。关联：非负频数才能按前缀找第 k 个；一般聚合或复杂更新用线段树

C++ 模板 · Fenwick

外部 add 的 $i \in [0, n)$ ，sum 要求 $0 \leq l \leq r \leq n$ 。所有累计结果须在 long long 范围内。

```
struct Fenwick {
    int n; vector<long long> bit;
    explicit Fenwick(int n) : n(n), bit(n + 1, 0) {}
    void add(int i, long long delta) {
        for (++i; i <= n; i += i & -i) bit[i] += delta;
    }
    long long prefix(int r) const { // [0, r)
        long long ans = 0;
        for (; r > 0; r -= r & -r) ans += bit[r];
        return ans;
    }
    long long sum(int l, int r) const {
        return prefix(r) - prefix(l);
    }
};
```

3.08 Segment Tree 与 Sparse Table **

思维关联 T5 分解与组合 · T4 复用计算 · T6 充分摘要

分块组织聚合，并复用已有子区间结果

解决什么 / 识别信号 维护一般区间聚合或静态 RMQ；动态最小值/最大值/和；静态大量区间最值

核心思想 线段树要求合并满足结合律，并使用单位元；Sparse Table 存 2^k 长度区间，RMQ 可用重叠两块

复杂度 / 不变量 线段树建 $O(n)$ 、点改/查询 $O(\log n)$ 、空间 $O(n)$ ；Sparse Table 建/空间 $O(n \log n)$ ，幂等操作查询 $O(1)$ 。不变量：父节点是子区间的正确合并；Sparse Table 每项覆盖定义的长度

常见坑 / 替代方案 普通 Sparse Table 的 $O(1)$ 重叠查询要求幂等，不能直接求和；非交换合并必须保序。关联：Lazy propagation * 处理范围更新；Disjoint Sparse Table * 可支持一般静态结合操作

C++ 模板 · Iterative Segment Tree

这是“和”的模板； $n \leq \text{INT_MAX}/2$ ，聚合不溢出。setValue 只对非空树的合法位置调用。非交换操作查询须分别累积左右结果再合并。

```
struct SegTree {
    int n; vector<long long> t;
    explicit SegTree(const vector<long long>& a)
        : n((int)a.size()), t(2 * n, 0) {
        for (int i = 0; i < n; ++i) t[n + i] = a[i];
        for (int i = n - 1; i > 0; --i)
            t[i] = t[i * 2] + t[i * 2 + 1];
    }
    void setValue(int i, long long x) {
        for (t[i += n] = x; i > 1; i /= 2)
            t[i / 2] = t[i] + t[i ^ 1];
    }
    long long sum(int l, int r) const { // [l, r)
        long long ans = 0;
        for (l += n, r += n; l < r; l /= 2, r /= 2) {
            if (l & 1) ans += t[l++];
            if (r & 1) ans += t[--r];
        }
        return ans;
    }
};
```

区间数据结构比较表 *** / ** / *

结构与星级	更新能力	查询 / 前提	构建 / 空间
Prefix Sum ***	改一个值通常重建	静态和 $O(1)$	$O(n)$ / $O(n)$
Difference ***	范围加 $O(1)$	最终恢复 $O(n)$ ，非在线	$O(n)$ / $O(n)$
Fenwick **	单点加 $O(\log n)$	前缀/范围和 $O(\log n)$	$O(n)$ 可行 / $O(n)$
两个 Fenwick **	范围加 $O(\log n)$	范围和 $O(\log n)$	$O(n)$ / $O(n)$
Segment Tree **	点改 $O(\log n)$	结合聚合 $O(\log n)$	$O(n)$ / $O(n)$
Lazy Segment Tree *	合法范围变换 $O(\log n)$	范围聚合 $O(\log n)$	$O(n)$ / $O(n)$
Sparse Table **	静态	幂等 RMQ $O(1)$	$O(n \log n)$ / 同阶
分块 *	按块懒标记或局部修改	常见 $O(\sqrt{n})$ ，视操作而定	通常 $O(n)$ / $O(n)$

选型顺序：静态可逆聚合 → 前缀；批量范围更新、最终读取 → 差分；在线加法 → Fenwick；一般动态聚合 → 线段树；静态幂等 RMQ → Sparse Table。操作代数比“数据量大”更能决定结构。

用具体操作检验选型 **

题目操作	选择原因
数组固定，10 万次区间求和	Prefix：一次 $O(n)$ 预处理，之后每次 $O(1)$ 。
先做 10 万次区间加，再输出整个数组	Difference：每次 $O(1)$ ，最后 $O(n)$ 恢复。
单点改值与区间和交错	Fenwick 存增量，先用 old value 算 delta；Segment Tree 也可。
区间最小值与单点改值交错	Segment Tree；普通加法 Fenwick 的前缀差不能恢复任意区间 min。
区间赋值与区间求和交错	Lazy Segment Tree：节点和改为 $value \times 区间长度$ ，赋值标记覆盖旧赋值。

两个 Fenwick 支持范围加、范围和 **

只在本公式中使用 1-based 闭区间 $[l,r]$ 。维护差分树 B1 和加权差分树 B2：B1 在 l 加 d 、 $r+1$ 加 $-d$ ；B2 在 l 加 $d(l-1)$ 、 $r+1$ 加 $-dr$ 。前 x 项之和为 $x \cdot query(B1,x) - query(B2,x)$ ，任意区间再作两个前缀之差。

推导抓手：每个差分 $d[i]$ 对前 x 项的贡献是 $d[i] \cdot (x-i+1)$ 。拆成 $x \cdot \sum d[i] - \sum d[i](i-1)$ ，就得到两棵树。 $r=n$ 时的 $r+1$ 是边界外标记，可省略更新；乘积与累计和都要检查位宽。

04 核心算法

把排序、选择、二分作为可复用基础动作；先说明契约，再讨论如何嵌入更大的模型。

4.01 排序与计数 / 桶思维 ***

思维关联 T8 调整处理顺序 · T6 充分摘要

重排暴露顺序；频数桶保留值的分布

解决什么 / 识别信号 建立顺序以简化扫描、去重和贪心；无序数据但顺序有价值；值域很小

核心思想 `std::sort` 通常优先；归并排序便于稳定合并与跨区间统计；小值域可用计数，桶用于分布或频率分组

复杂度 / 不变量 `sort` $O(n \log n)$ 比较；归并 $O(n \log n)/O(n)$ ；计数 $O(n+U)/O(U)$ 。不变量：排序后相邻次序满足同一比较关系

常见坑 / 替代方案 比较器用 `<=` 破坏严格弱序；排序可能破坏原索引；小 n 大 U 不适合计数。关联：保留 `(value, index)`；需稳定用 `stable_sort`；排序性质见第 15 章

4.02 Quickselect、Top K 与 Partition ***

思维关联 T2 排除候选 · T6 充分摘要

舍弃不含目标秩的部分或只保留 K 个候选

解决什么 / 识别信号 找秩或分出前 K 个，无需全部排序；第 k 小、前 K 集合、数据流最值

核心思想 `nth_element` 只保证分区秩；大小 K 堆维护流式候选；三路划分把重复 `pivot` 一次分完

复杂度 / 不变量 `nth_element` 非并行版本平均 $O(n)$ ；随机 Quickselect 期望 $O(n)$ 、最坏 $O(n^2)$ ；堆 $O(n \log K)$ 。不变量：三路划分中 `<p`、`==p`、未知、`>p` 四段持续成立

常见坑 / 替代方案 第 k 小是下标 $k-1$ ； $K=0$ 需单独处理；划分后两侧内部无序。关联：前 K 还要有序用 `partial_sort` 或选择后排序；Lomuto ** 简单、Hoare ** 必须匹配自己的边界递归

C++ 模板 · Three-way Partition

划分整个数组，可包含重复值；从右边换入的未知元素仍需检查，所以第二个分支不递增 `i`。

```
pair<int,int> partition3(vector<int>& a, int pivot) {
    int lo = 0, i = 0, hi = (int)a.size();
    while (i < hi) {
        if (a[i] < pivot) swap(a[lo++], a[i++]);
        else if (a[i] > pivot) swap(a[i], a[--hi]);
        else ++i;
    }
    return {lo, hi}; // == pivot in [lo, hi)
}
```

4.03 边界二分与精确查找 ***

思维关联 T2 排除候选

依据单调判定排除不可能的候选区间

解决什么 / 识别信号 找到有序边界或单调布尔状态的转折；lower_bound、upper_bound、first true

核心思想 维护待定半开区间 [lo,hi)；判定为 true 时缩右边界，否则越过 mid。精确查找是边界定位后检查相等

复杂度 / 不变量 $O(\log n)$ 次判定、 $O(1)$ 空间；总时间乘以 predicate 成本。不变量：[0,lo) 全 false，[hi,n) 全 true，未决定区间为 [lo,hi)

常见坑 / 替代方案 返回 n 表示不存在；对链表使用通用 lower_bound 迭代器移动可能 $O(n)$ 。关联：有序容器使用成员 lower_bound；非单调条件不可套用

C++ 模板 · First True

$n \geq 0$ ；ok 在 [0,n) 上先 false 后 true，可全 false 或全 true。不会调用 ok(n)。

```
template<class Pred>
int firstTrue(int n, Pred ok) {
    int lo = 0, hi = n;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (ok(mid)) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
```

4.04 Binary Search on Answer ***

思维关联 T1 转换与建模 · T2 排除候选

优化转判定，再由单调性排除答案域

解决什么 / 识别信号 把难以直接优化的答案转成可行性判断；最小化最大负载、最大化最小间距、最小速度

核心思想 明确答案域、判定函数、单调方向、可行边界。先证明“更宽松的答案不会更难可行”，再二分

复杂度 / 不变量 $O(\text{check} \times \log R)$ ，R 是整数答案域宽；空间取决于 check。不变量：每轮保留所有可能的最优可行答案，且预设的可行边界不丢失

常见坑 / 替代方案 仅凭 min/max 措辞不能证明单调；用错误贪心检查会让二分稳定地产生错误答案。关联：如果状态很小，可直接 DP；实数答案按误差/固定迭代次数二分

示例：分成不超过 K 个非空连续段，最小化最大段和 ***

前提：数组非空且 $a[i] \geq 0$ ， $1 \leq K \leq n$ 。下界 $\max(a)$ ，上界 $\text{sum}(a)$ 。给定 cap，从左到右尽量装满当前段；放不下才开新段，得到所需最少段数。cap 增大时所需段数不增加，因此“段数 $\leq K$ ”是 false $\rightarrow$ true。若题目要求恰好 K 段，非负元素保证可继续拆分而不增加最大段和。负数输入下此贪心和边界均需重新分析。

C++ 模板 · Minimize Maximum Segment Sum

满足上述输入前提，并保证总和可表示。check $O(n)$ ，总时间 $O(n \log(\text{sum} - \text{max} + 1))$ ，额外空间 $O(1)$ 。

```
long long minLargestSum(const vector<int>& a, int K) {
    long long lo = *max_element(a.begin(), a.end());
    long long hi = accumulate(a.begin(), a.end(), 0LL);
    auto ok = [&](long long cap) {
        int groups = 1; long long cur = 0;
        for (int x : a) {
            if (cur > cap - x) { ++groups; cur = x; }
            else cur += x;
        }
        return groups <= K;
    };
    while (lo < hi) {
        long long mid = lo + (hi - lo) / 2;
        if (ok(mid)) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
```

4.05 结构化二分 **

思维关联 T2 排除候选 · T1 转换与建模

先识别有序片段或方向，再排除候选

解决什么 / 识别信号 在部分有序或局部结构中缩小范围；旋转有序数组、峰值、行列有序矩阵

核心思想 旋转数组先识别有序的一半；峰值利用邻居方向保留至少一个峰；矩阵可对值域计数后搜答案或阶梯扫描

复杂度 / 不变量 无重复旋转搜索/峰值 $O(\log n)$ ；有重复旋转最坏 $O(n)$ ；矩阵阶梯 $O(r+c)$ 。不变量：每次删除的一半不含目标，或保留区间中仍存在一个目标峰

常见坑 / 替代方案 平台与重复值破坏严格判别；任意函数不能当作单峰；二维全局有序与行列分别有序不同。关联：单峰函数三分*；可直接用有序矩阵堆选第 k 小

二分的边界语义速查 ***

目标	谓词或转换	不存在时
第一个 $\geq x$	<code>firstTrue(n, a[i] >= x)</code>	返回 n
第一个 $> x$	<code>firstTrue(n, a[i] > x)</code>	返回 n
最后一个 $\leq x$	<code>upper_bound(x)</code> 的位置减 1	先用有符号索引，可能为 -1
精确查找 x	先找 <code>lower_bound</code> ，再检查位置 $< n$ 且值 $== x$	按题意返回 -1/false
最大可行整数	谓词 <code>true</code> $\rightarrow$ <code>false</code> ，可找首个不可行再减 1	先处理全不可行和全可行边界
实数答案	保持可行/不可行两端，按精度或固定轮数结束	返回约定的一侧并说明误差

旋转数组例 [2,2,2,3,2]：仅看首尾相等无法判断哪半有序，逐步缩小重复边界可能退化到线性。这不是代码写得不够巧，而是重复值减少了可获得的判别信息。 **

二分落笔前五问 ***：谓词是否单调？答案域是什么？哪一端已知可行？每轮区间是否严格缩小？全部 `true`/全部 `false` 时返回什么？对每个分支写出这五个答案，比背多个 `while` 模板更可靠。

05 线性序列题型

连续区间、指针推进、前缀状态、单调结构、区间和堆题族组成数组与字符串题的主干。

5.01 双指针的五种角色 ***

思维关联 T2 排除候选 · T8 调整处理顺序 · T4 复用计算

安全跳过候选，按顺序推进并维护局部变化

解决什么 / 识别信号 利用有序性或单向推进减少枚举；两数和、合并序列、压缩数组、链表 midpoint

核心思想 相向指针排除不可能配对；同向指针维护窗口；读写指针稳定压缩；双流指针合并；快慢指针描述追赶或间距

复杂度 / 不变量 常见 $O(n+m)$ 时间、 $O(1)$ 辅助空间；排序前置成本另计。不变量：已越过区域已完成，指针只朝允许方向移动

常见坑 / 替代方案 未排序数组直接相向找两数和；写指针覆盖尚未读的数据；重复值去重语义不清。关联：两数和可用哈希；需要全体配对时输出本身可能 $O(n^2)$

5.02 滑动窗口与 exactly K ***

思维关联 T4 复用计算 · T2 排除候选 · T7 贡献与抵消

复用窗口统计，排除非法左端并累计合法区间

解决什么 / 识别信号 求连续区间的长度、计数或可维护统计；固定窗口；“至多 K 类”；非负数和的单调约束

核心思想 加入右端，必要时收缩左端恢复合法性。若固定右端的合法左端形成连续后缀，则贡献 $\text{right} - \text{left} + 1$ 。
 $\text{exactly } K = \text{atMost}(K) - \text{atMost}(K-1)$

复杂度 / 不变量 每个端点最多推进 n 次，通常 $O(n)$ ；统计空间依值域。不变量：缩完后窗口合法，且 left 是当前右端的最早合法起点（按题型证明）

常见坑 / 替代方案 任意负数的和不具备常规窗口单调性；计数用 long long； $K < 0$ 的 atMost 应返回 0。关联：负数和等值计数用 Prefix+Hash；最短和至少 K 可用前缀+单调队列 **

C++ 模板 · At Most K Distinct

统计非空连续子数组，哈希操作按平均复杂度计。恰好 K 类的数量为 $\text{atMost}(a, K) - \text{atMost}(a, K-1)$ ，要求 $K \geq 0$ 。

```
long long atMostK(const vector<int>& a, int K) {
    if (K < 0) return 0;
    unordered_map<int, int> cnt;
    long long ans = 0; int left = 0;
    for (int right = 0; right < (int)a.size(); ++right) {
        ++cnt[a[right]];
        while ((int)cnt.size() > K) {
            int x = a[left++];
            if (--cnt[x] == 0) cnt.erase(x);
        }
        ans += right - left + 1LL;
    }
    return ans;
}
```

5.03 Prefix State: 统一前缀模型 ***

思维关联 T1 转换与建模 · T6 充分摘要 · T7 贡献与抵消
区间转两前缀关系，用状态消去公共历史

解决什么 / 识别信号 把区间条件转为两个前缀状态关系；子数组和/XOR、整除、平衡计数、字符奇偶

核心思想 定义 $S[0]$ 为空前缀，区间 $[i,j]$ 用 $S[j]$ 与 $S[i]$ 消去公共部分。等值计数存频次，最长长度存最早位置，最短长度按目标存最近位置或有序候选

复杂度 / 不变量 常见 $O(n)$ 期望时间、 $O(n)$ 空间；小状态域可降到固定空间。不变量：扫描 j 时，容器只保存 $i < j$ 的前缀，查询后才插入当前状态

常见坑 / 替代方案 忘记空前缀；先插入后查询导致多计空区间；频次与最早索引混用。关联：窗口依赖单调性；Prefix State 依赖可组合状态及区间关系

前缀状态与星级	区间条件 → 查历史	关键边界
Sum ***	$sum(i,j)=K \rightarrow S[i]=S[j]-K$	$freq[0]=1$ ；支持负数
XOR ***	$xor(i,j)=K \rightarrow X[i]=X[j]^K$	$X[0]=0$ ；求和与 XOR 不能互换
Remainder ***	$sum(i,j)\%k=0 \rightarrow R[i]=R[j]$	$k>0$ ，负余数归一化
Balance ***	$0 \rightarrow -1, 1 \rightarrow +1 \rightarrow$ 两端状态相等	最长长度保存首次出现位置；空前缀位置 -1
Parity Mask **	字符次数全偶数 $\rightarrow$ mask 相同	每来一个字符翻转对应位
Palindrome parity **	至多一位为 1 $\rightarrow$ 同 mask 或差一位	字母表 σ ：每步查 $\sigma+1$ 个状态

C++ 模板 · Prefix Sum Count

要求前缀和、sum-K 和答案均可表示。例 $a=[1,-1,1]$, $K=1$ ，结果 3；负数不会影响正确性。

```
long long countSumK(const vector<int>& a, long long K) {
    unordered_map<long long, long long> freq;
    freq[0] = 1;
    long long sum = 0, ans = 0;
    for (int x : a) {
        sum += x;
        auto it = freq.find(sum - K);
        if (it != freq.end()) ans += it->second;
        ++freq[sum];
    }
    return ans;
}
```

5.04 区间合并、交集与调度 ***

思维关联 T8 调整处理顺序 · T6 充分摘要 · T2 排除候选

排序后汇总覆盖，或证明安全的贪心选择

解决什么 / 识别信号 归并覆盖、两组交集、最大兼容区间数；会议、时间段、覆盖与选取

核心思想 合并按左端排序并延长右端；两个有序不相交区间列表用双指针求交；最大不重叠数量按结束时间贪心

复杂度 / 不变量 排序 $O(n \log n)$ ，后续扫描 $O(n)$ ；两组交集 $O(n+m)$ 。不变量：合并结果内部不相交；调度已选集合可行且末端尽可能早

常见坑 / 替代方案 半开区间端点相同不算重叠；能否合并相邻区间由题意决定；加权调度不能用简单最早结束贪心。关联：加权调度用排序+二分+DP；最少区间覆盖用当前可达范围内最远延伸

5.05 扫描线、事件与差分映射 ***

思维关联 T8 调整处理顺序 · T4 复用计算 · T7 贡献与抵消

按事件推进，只更新活跃状态的变化

解决什么 / 识别信号 将重叠和变化统一为有序事件；会议室数量、并发峰值、活跃区间

核心思想 把开始/结束变成事件，排序后维护活跃状态；同坐标事件按题意分组或设优先级。二维扫描线再配范围结构

复杂度 / 不变量 一维常见 $O(n \log n)/O(n)$ ；二维矩形并面积通常 $O(n \log n)$ 。不变量：处理到坐标 x 时，活跃集合精确对应当前扫描位置

常见坑 / 替代方案 $[l, r)$ 的同点结束先于开始；闭区间不同；矩形并面积不能只加长度，要乘坐标差。关联：整数密集坐标可差分；区间调度是选择问题，不是重叠计数

5.06 单调栈与贡献法 ***

思维关联 T2 排除候选 · T7 贡献与抵消

解决或排除候选，并按负责区间统计贡献

解决什么 / 识别信号 找最近更大/小元素或每个值的贡献范围；下一个更大、直方图、子数组最小值之和

核心思想 栈存待解决的下标，遇到破坏单调性的值就结算弹出元素；最小值贡献为 $a[i] \times \text{左可选数} \times \text{右可选数}$

复杂度 / 不变量 每个索引进出一次， $O(n)$ 时间、 $O(n)$ 空间。不变量：栈中下标递增，对应值按指定严格性单调

常见坑 / 替代方案 相等元素两边都严格/都非严格会漏计或重计；直方图弹栈后宽度取新栈顶。关联：贡献计数常用左严格小、右小于等于（或对称方案）；RMQ 是不同模型

C++ 模板 · Next Greater Index

返回右侧第一个严格更大元素的下标；不存在为 -1。相等元素不会互相弹出。

```
vector<int> nextGreater(const vector<int>& a) {
    int n = (int)a.size();
    vector<int> ans(n, -1), st;
    for (int i = 0; i < n; ++i) {
        while (!st.empty() && a[st.back()] < a[i]) {
            ans[st.back()] = i;
            st.pop_back();
        }
        st.push_back(i);
    }
    return ans;
}
```

5.07 单调队列 ***

思维关联 T2 排除候选 · T4 复用计算

删除被支配候选，增量维护窗口最优值

解决什么 / 识别信号 维护滑动窗口最值或前缀候选；每个长度 K 窗口的最大值

核心思想 队首移除过期索引，队尾移除被当前元素支配的候选；较晚且不小的值比旧值更有用

复杂度 / 不变量 $O(n)$ 时间、 $O(K)$ 空间。不变量：下标递增、值递减；队首是窗口内最优候选

常见坑 / 替代方案 仅按值删除无法识别过期；窗口 $K=0$ 或 $K>n$ 需约定；队列不等于堆。关联：需要任意动态删除或中位数时换树/双堆

C++ 模板 · Sliding Maximum

队尾删除等值旧元素，保留更晚过期的新元素。

```
vector<int> slidingMax(const vector<int>& a, int K) {
    if (K <= 0 || K > (int)a.size()) return {};
    deque<int> q; vector<int> ans;
    for (int r = 0; r < (int)a.size(); ++r) {
        while (!q.empty() && q.front() <= r - K)
            q.pop_front();
        while (!q.empty() && a[q.back()] <= a[r])
            q.pop_back();
        q.push_back(r);
        if (r + 1 >= K) ans.push_back(a[q.front()]);
    }
    return ans;
}
```

5.08 Heap Patterns: Top K 与 K-way Merge ***

思维关联 T6 充分摘要 · T8 调整处理顺序

只放每路下一候选，按优先级产生结果

解决什么 / 识别信号 从少量当前候选中逐次取最优；K 个有序流、排名、任务释放与执行

核心思想 Top K 最大值用大小 K 的最小堆；K 路合并每路只放当前最小未输出项，弹出后推进该路

复杂度 / 不变量 Top K $O(n \log K)/O(K)$ ；K 路合并 N 项 $O(N \log K)/O(K)$ 。不变量：每路的堆中候选代表该路未来最小值；堆中总是所有可能的下一个输出

常见坑 / 替代方案 把所有 N 项一起放堆会变为 $O(N \log N)$ ；值相同需确定 tie-break； $K=1$ 退化为线性。关联：静态 Top K 可 nth_element；调度可能需要“等待堆+就绪堆”

5.09 双堆中位数与惰性删除 **

思维关联 T6 充分摘要 · T8 调整处理顺序 · T4 复用计算

维护有序分隔，延迟清理失效状态

解决什么 / 识别信号 在线中位数和滑动中位数；不断插入；窗口过期元素不一定在堆顶

核心思想 小半边最大堆 L，大半边最小堆 R；保持 $|L|=|R|$ 或 $|L|=|R|+1$ ，且 $\max(L) \leq \min(R)$ 。删除用待删计数和逻辑大小，清理堆顶后再平衡

复杂度 / 不变量 仅插入 $O(\log n)$ 、读中位 $O(1)$ ；滑窗用 multiset 可 $O(n \log K)/O(K)$ 。不变量：逻辑大小和两侧有序分割成立，取 top 前先清理过期项

常见坑 / 替代方案 惰性堆的物理大小可能累积到 $O(n)$ ，不能无条件声称 $O(K)$ 空间；均值相加可能溢出。关联：窗口双堆可周期重建；面试常用两个 multiset 更直接

5.10 链表反转、Dummy 与快慢指针 ***

思维关联 T1 转换与建模 · T8 调整处理顺序
Dummy 统一边界；后继关系与推进节奏暴露环

解决什么 / 识别信号 在 $O(1)$ 额外空间下改变链接或识别环；删除头节点、合并、区间反转、中点、倒数 K 个
核心思想 Dummy 统一头部处理；反转先保存 next；Floyd 快慢指针相遇后，一指针回起点，二者同速得到环入口
复杂度 / 不变量 反转/判环 $O(n)$ 时间、 $O(1)$ 空间。不变量：已反转前缀与未处理后缀互不丢失；快慢指针位于同一后继轨道
常见坑 / 替代方案 先改 next 再读会丢链；偶数长度中点约定不一致；fast->next 访问前判空。关联：HashSet 判环 $O(n)$ 空间；数组后继、Happy Number 也可用 Floyd

C++ 模板 · Reverse List

输入必须是有限无环链表；函数修改链接，不创建或释放节点。

```
struct ListNode { int value; ListNode* next; };
ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    while (head != nullptr) {
        ListNode* next = head->next;
        head->next = prev;
        prev = head;
        head = next;
    }
    return prev;
}
```

链表反转的指针推演 ***

状态	prev: 已反转前缀	head: 待处理后缀
初始	空	1 → 2 → 3 → 空
处理 1 后	1 → 空	2 → 3 → 空
处理 2 后	2 → 1 → 空	3 → 空
处理 3 后	3 → 2 → 1 → 空	空, 返回 prev

每次先把 head->next 存在局部变量 next，再修改链接。循环结束时 head 为空、prev 就是新头。若要反转 $[l,r)$ 的一段，还需保存段前驱与原段首，并在局部反转完成后接回两侧；Dummy 能统一 $l=0$ 的情况。

Floyd 环入口为什么成立 ***

设进入环前长度为 μ 、环长为 λ 。快慢指针相遇时，慢指针已走 t 步，快指针已走 $2t$ 步；环内相遇意味着 t 是 λ 的倍数。将一个指针放回起点，另一个留在相遇点，两者同速再走 μ 步，前者到入口，后者的环内位置也回到入口。先判定存在环，才能进行这一步。

中点与倒数 K 个 ***: fast 每次两步、slow 每次一步，常见循环在偶数长度时返回第二个中点；若要第一个中点，应更改起始位置或终止条件。倒数 K 个可先让 fast 领先 K 步，再同步推进；删除时用 Dummy 并核对 K 的合法范围。

06 树算法

树的核心是父子结构与子问题边界：向下传什么、向上返回什么，以及何时可以把树展平为区间。

6.01 遍历、递归返回值与 BST ***

思维关联 T5 分解与组合 · T8 调整处理顺序 · T6 充分摘要

按信息依赖遍历，向父节点返回必要摘要

解决什么 / 识别信号 组织树信息的流动与有序检索；树高、路径、验证 BST、层次统计

核心思想 前序传父状态，后序合并子状态，中序利用 BST 顺序，BFS 按层。递归先定义返回值，区分“以此节点向下的最优”与“整棵子树全局最优”

复杂度 / 不变量 遍历 $O(n)$ ；DFS 栈 $O(h)$ ，BFS $O(\text{最大层宽})$ ；BST 操作 $O(h)$ 。不变量：每个返回值符合统一含义；BST 的整个子树满足上下界

常见坑 / 替代方案 只比父子无法验证 BST；重复值规则需明确；退化树递归栈溢出。关联：深树用显式栈；验证 BST 可用可选上下界或中序前驱

6.02 LCA 与 Binary Lifting **

思维关联 T4 复用计算 · T5 分解与组合

复用短跳跃，组合目标距离

解决什么 / 识别信号 多次公共祖先或第 k 个祖先查询；树上多次路径关系；很大的跳步次数

核心思想 普通递归 LCA *** 用左右结果汇合，单次 $O(n)$ ；倍增 $up[v][j]$ 表示 2^j 步祖先，先拉齐深度，再从大步到小步同时上跳

复杂度 / 不变量 预处理 $O(n \log n)/O(n \log n)$ ，查询 $O(\log n)$ ；单次递归空间 $O(h)$ 。不变量：每次跳后答案祖先不被越过；同步跳后两点仍位于 LCA 的不同后代分支

常见坑 / 替代方案 普通 LCA 默认目标存在，否则应额外计数验证；根祖先和 -1 哨兵混用。关联：Euler 完整游走 + 深度 RMQ；倍增也可用于函数迭代、路径聚合

6.03 Euler Tour / Tree Flattening **

思维关联 T1 转换与建模 · T8 调整处理顺序

以 DFS 顺序把子树转换为连续区间

解决什么 / 识别信号 把子树查询转换成数组区间；子树求和、子树加值、祖先判定

核心思想 DFS 首次进入给 $tin[v]$ ，遍历完全部子树给 $tout[v]$ ，每点只计一次。子树恰为 $[tin[v], tout[v]]$ 。节点值按 tin 重排后接 Fenwick/线段树

复杂度 / 不变量 展平 $O(n)/O(n)$ ；子树查询由范围结构决定。不变量：DFS 在离开节点前会连续完成它的全部后代

常见坑 / 替代方案 子树序列长 n ，LCA 用的进入/回退完整游局长 $2n-1$ ，两者不能混用；无向树必须跳过父边。关联：节点到根路径可结合差分；一般路径更新/查询用 HLD *

C++ 模板 · Iterative Tree Flattening

输入为非空连通无向树、合法根。该模板只跳父节点，不用于有环图。迭代实现避免深树递归溢出。

```
pair<vector<int>, vector<int>> flattenTree(
    const vector<vector<int>>& g, int root) {
    int n = (int)g.size(), timer = 0;
    vector<int> tin(n), tout(n), parent(n, -1), next(n);
    vector<int> st{root};
    parent[root] = root; tin[root] = timer++;
    while (!st.empty()) {
        int v = st.back();
        if (next[v] == (int)g[v].size()) {
            tout[v] = timer; st.pop_back();
        } else {
            int u = g[v][next[v]++];
            if (u == parent[v]) continue;
            parent[u] = v; tin[u] = timer++;
            st.push_back(u);
        }
    }
    return {tin, tout};
}
```

小例子：0 的孩子为 1、2，1 的孩子为 3、4。按 1 再 2 遍历，序列为 [0,1,3,4,2]，节点 1 的子树区间是 [1,4)。祖先判定：
 $\text{tin}[u] \leq \text{tin}[v] < \text{tout}[u]$ 。

6.04 Tree DP、直径与换根 **

思维关联 T5 分解与组合 · T4 复用计算 · T6 充分摘要

合并子树摘要，换根时复用已有贡献

解决什么 / 识别信号 对所有子树或所有根计算最优统计；树上独立集、直径、到所有节点距离和

核心思想 树 DP 后序合并孩子状态。非负边权直径可两次最远点遍历；一般权值用后序路径 DP。换根 * 先算子树，再将父侧贡献传给孩子

复杂度 / 不变量 普通树 DP 常 $O(n)$ ，背包式合并可能更高；无权距离和换根 $O(n)$ 。不变量：合并时只使用互不相交的孩子子问题；换根必须排除目标孩子已有贡献

常见坑 / 替代方案 把两次 BFS 套到任意带负权树；直径按边数还是节点数需一致。关联：无权距离和公式
 $\text{ans}[\text{child}] = \text{ans}[\text{parent}] + n - 2 * \text{size}[\text{child}]$ ；带权边乘该边权

07 图算法

先识别图的结构与边权，再选择遍历、顺序、连通或最短路；不同目标不能共用一个模糊的“图模板”。

7.01 表示、DFS / BFS 与隐式图 ***

思维关联 T1 转换与建模 · T8 调整处理顺序

先定义状态图，再按适当次序访问

解决什么 / 识别信号 搜索可达性、分量与等代价最短路径；网格、状态转换、关系网络

核心思想 稀疏图用邻接表，稠密图可矩阵，边列表便于排序/松弛。隐式图按需生成邻居。BFS 入队时标记，DFS 进入时标记

复杂度 / 不变量 邻接表 $O(V+E)$ 时间/空间；矩阵遍历 $O(V^2)$ 。不变量：BFS 首次发现距离最短（单位边权）；每个状态按正确身份去重

常见坑 / 替代方案 只用坐标作 visited 却遗漏钥匙/剩余资源状态；无向边要加两次；网格面积可能很大。关联：递归太深用显式栈；不连通图从每个未访问点启动

7.02 多源 BFS ***

思维关联 T1 转换与建模 · T8 调整处理顺序

多个起点统一为同一分层传播过程

解决什么 / 识别信号 一次计算到最近源的距离；最近设施、腐烂扩散、同时发生的传播

核心思想 所有源以距离 0 同时入队，相当于连到超级源但不用真的建边；随后正常 BFS

复杂度 / 不变量 $O(V+E)$ 时间、 $O(V)$ 辅助空间。不变量：队列按离任意源的最短距离分层

常见坑 / 替代方案 依次对每个源做 BFS 再取最小会重复工作；有障碍时曼哈顿距离不够。关联：有非负不同边权时改多源 Dijkstra；双向 BFS 见第 12 章

C++ 模板 · Multi-source BFS

所有源和邻接点下标合法；不可达返回 -1。适用于无权/单位权图。

```
vector<int> multiBfs(const vector<vector<int>>& g,
                    const vector<int>& sources) {
    vector<int> d(g.size(), -1); queue<int> q;
    for (int s : sources) {
        if (d[s] == -1) { d[s] = 0; q.push(s); }
    }
    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (int u : g[v]) if (d[u] == -1) {
            d[u] = d[v] + 1;
            q.push(u);
        }
    }
    return d;
}
```

7.03 拓扑排序、三色 DFS 与 DAG ***

思维关联 T8 调整处理顺序
按依赖顺序计算，并用活动路径识别环

解决什么 / 识别信号 安排依赖顺序、检测有向环、DAG 上 DP；任务先后、课程先修、无环依赖

核心思想 Kahn 删除入度为 0 的点；三色 DFS：0 未访问、1 当前路径、2 已完成，指向 1 的边说明有环；DAG DP 按拓扑顺序传播

复杂度 / 不变量 $O(V+E)$ 时间、 $O(V)$ 辅助空间。不变量：Kahn 队列节点在剩余图中无前驱；DFS 的颜色 1 正好对应递归栈

常见坑 / 替代方案 遇到任意 visited 不等于遇环；有环时无完整拓扑序；最长路仅在 DAG 等结构上容易。关联：字典序最小拓扑序用小根堆；无向判环用父边/DSU

C++ 模板 • Kahn Topological Sort

返回空序列可表示有环，也可能是空图；调用者结合 n 区分。平行边需按实际边数统计入度。

```
vector<int> topoSort(const vector<vector<int>>& g) {
    int n = (int)g.size(); vector<int> in(n), order;
    for (const auto& adj : g) for (int u : adj) ++in[u];
    queue<int> q;
    for (int v = 0; v < n; ++v) if (in[v] == 0) q.push(v);
    while (!q.empty()) {
        int v = q.front(); q.pop(); order.push_back(v);
        for (int u : g[v]) if (--in[u] == 0) q.push(u);
    }
    if ((int)order.size() != n) return {};
    return order;
}
```

最短路决策表 *** / **

图与边权	算法	时间 / 关键限制
所有边相同正权 / 无权	BFS ***	$O(V+E)$ ；按层最短
仅 0 和 1	0-1 BFS **	$O(V+E)$ ；0 权压队首，1 权压队尾
DAG，允许负权	拓扑松弛 **	$O(V+E)$ ；先确认无环
一般非负权	Dijkstra ***	二叉堆常 $O((V+E)\log V)$ ；见下文实现界限
有负边，单源	Bellman-Ford **	$O(VE)$ ；可检测源可达负环
小图，全点对	Floyd-Warshall **	$O(V^3)$ / $O(V^2)$ ；允许负边，检查负环
稀疏图，全点对	重复 Dijkstra / Johnson *	非负权可重复；有负边用重赋权，要求无负环

7.04 Dijkstra ***

思维关联 T8 调整处理顺序 · T2 排除候选

安全定案最小距离，丢弃较差或过期状态

解决什么 / 识别信号 一般非负权图单源最短路；路程、花费、非负状态转移

核心思想 每次取暂定距离最小点，松弛出边；C++ priority_queue 用新状态入堆，弹出时跳过过期项

复杂度 / 不变量 惰性二叉堆 $O(V+E \log(E+1))$ ，堆空间 $O(E)$ ；简单图常简写 $O((V+E)\log V)$ 。不变量：弹出的当前有效最小距离已经是最终最短距离

常见坑 / 替代方案 负权破坏证明；visited 不能在入堆时定案；INF 加边权会溢出。关联：稠密图朴素 $O(V^2+E)$ 可更合适；0/1 权用 deque [R2]

C++ 模板 · Dijkstra

所有边权非负、节点合法；所有需要区分的有限最短距离必须严格小于 INF。否则应换数值表示，不能把截断结果当作精确距离。

```
struct Edge { int to; long long w; };
vector<long long> dijkstra(const vector<vector<Edge>>& g,
                          int source) {
    const long long INF = numeric_limits<long long>::max()/4;
    vector<long long> d(g.size(), INF);
    using State = pair<long long,int>;
    priority_queue<State, vector<State>, greater<State>> pq;
    d[source] = 0; pq.emplace(0, source);
    while (!pq.empty()) {
        auto [du, v] = pq.top(); pq.pop();
        if (du != d[v]) continue;
        for (auto [u, w] : g[v]) {
            if (w > INF - du) continue;
            long long nd = du + w;
            if (nd < d[u]) {
                d[u] = nd; pq.emplace(nd, u);
            }
        }
    }
    return d;
}
```

7.05 0-1 BFS、Bellman-Ford 与 Floyd **

思维关联 T8 调整处理顺序 · T4 复用计算

按权重、边数或中间点组织可复用的松弛

解决什么 / 识别信号 处理特殊边权、负权或全点对需求；Dijkstra 前提不满足或查询覆盖所有源

核心思想 0-1 BFS 按权重插入双端队列并在改善时松弛；Bellman-Ford 做 $V-1$ 轮边松弛，第 V 轮改善提示可达负环；Floyd 以中间点 k 为外层逐步扩展可用中间点

复杂度 / 不变量 分别 $O(V+E)$ 、 $O(VE)$ 、 $O(V^3)$ ；空间 $O(V)$ 、 $O(V)$ 、 $O(V^2)$ 。不变量：松弛不增加距离；Floyd 第 k 轮仅允许已处理的中间点

常见坑 / 替代方案 负环可达的相关路径没有有限最短值；Floyd 对 INF 相加前判可达；SPFA 最坏仍 $O(VE)$ 。关联：只需检测任意负环可加超级源；DAG 有负权优先拓扑松弛

7.06 MST: Kruskal 与 Prim **

思维关联 T2 排除候选 · T8 调整处理顺序

以割性质证明安全选边，保持可扩展的部分解

解决什么 / 识别信号 用最小总权连接所有点；修路、网络连通成本、无向加权图

核心思想 Kruskal 按边权排序，用 DSU 排除环；Prim 每步取跨当前割的最轻连接边

复杂度 / 不变量 Kruskal $O(E \log E)$ ；二叉堆 Prim 常 $O(E \log V)$ ，稠密版 $O(V^2)$ 。不变量：每步选边是某个割的安全边；已选集合是森林/连通树

常见坑 / 替代方案 MST 不是从源出发最短路径树；图不连通只能得最小生成森林；允许负权。关联：最小瓶颈、连通阈值可结合 Kruskal 离线；第二生成树属高阶扩展

7.07 SCC、桥、割点与双连通 **

思维关联 T3 合并等价状态 · T6 充分摘要 · T1 转换与建模

SCC 按可达目标缩点；low 摘要描述返祖能力

解决什么 / 识别信号 识别有向强连通与无向脆弱连接；互相可达、缩点、断一条边或一个点影响连通

核心思想 SCC 用 Kosaraju 或 Tarjan；缩点后为 DAG。无向 DFS 用 tin/low：树边 $v \rightarrow u$ 为桥当 $low[u] > tin[v]$ ；非根 v 为割点当某孩子 $low[u] \geq tin[v]$

复杂度 / 不变量 SCC/桥/割点 $O(V+E)$ 时间、 $O(V+E)$ 空间。不变量：low 表示经子树与允许的返祖边能到达的最早时间戳

常见坑 / 替代方案 根割点要求 DFS 孩子数 > 1 ；平行边用边 id 跳过父边；SCC 的栈规则不同于无向桥。关联：边双连通/点双连通 *；普通 DSU 不能直接维护有向 SCC

7.08 二分图与状态染色 **

思维关联 T1 转换与建模 · T6 充分摘要

把相邻约束表示为两种颜色的一致性

解决什么 / 识别信号 检测两组互斥关系或奇环；敌友约束、两类分配、匹配建模

核心思想 对每个分量 BFS/DFS 二染色，相邻点应异色；相同色冲突等价于存在奇环

复杂度 / 不变量 $O(V+E)$ 时间、 $O(V)$ 空间。不变量：已检查的每条边连接不同颜色

常见坑 / 替代方案 只从一个源出发会漏分量；有向边用于二分性质时通常按无向约束理解。关联：Parity DSU 可在线处理异同约束；匹配和最大流见第 13 章

7.09 Functional Graph: 每点一个后继 **

思维关联 T1 转换与建模 · T5 分解与组合 · T4 复用计算

分离入环路径与周期；数组转后继图推演见 T1

解决什么 / 识别信号 处理确定性迭代、环、跳 k 步、最终归宿；next[i]、传送、数组当链表、重复应用函数

核心思想 每个弱连通分量恰有一个有向环，其他点汇入该环。入度剥叶得到环点；沿环编号，再逆序处理被剥点的环入口与距离

复杂度 / 不变量 全图分解 $O(n)/O(n)$ ；倍增 $O(n \log K)$ 预处理、 $O(\log K)$ 查询。不变量：剥完后未删除点全在环上；逆序时后继的答案已知

常见坑 / 替代方案 允许“无后继”的图需另处理终止链；Floyd 只覆盖单条轨道，不能一轮得到所有分量。关联：单起点 $O(1)$ 空间判环用 Floyd；大量大步查询用倍增， K 必须覆盖位宽

08 算法设计范式

范式回答“如何设计解法”，并不与数据结构互斥：同一道题可以同时用分治、离线处理和 Fenwick。

8.01 分治与归并计数 ***

思维关联 T5 分解与组合 · T7 贡献与抵消 · T8 调整处理顺序

内部递归解决，有序合并统计跨边界贡献

解决什么 / 识别信号 将大问题拆成独立子问题，再统计跨边界关系；归并排序、逆序对、右侧更小数组、重要翻转对

核心思想 递归解决左右两半；两半有序后双指针统计跨区间贡献。逆序对遇到 $\text{left}[i] > \text{right}[j]$ 时，一次贡献 $\text{mid} - i$

复杂度 / 不变量 逆序计数 $O(n \log n)$ 时间、 $O(n)$ 缓冲和 $O(\log n)$ 栈。不变量：合并前两半有序，区间内答案已计入，只需新增跨区间贡献

常见坑 / 替代方案 严格/非严格比较影响重复值； $a[i] > 2 * a[j]$ 要先扩宽，不能直接照搬普通逆序归并判断。关联：Fenwick+离散化 ** 也可计数；分治递推 $T(n) = 2T(n/2) + O(n)$

C++ 模板 · Merge Sort Inversion Count

调用前 buf 大小至少 $a.size()$ ，调用 $\text{inversions}(a, \text{buf}, 0, a.size())$ 。函数会排序 a，统计严格逆序对。

```
long long inversions(vector<int>& a, vector<int>& buf,
                    int l, int r) {
    if (r - l <= 1) return 0;
    int m = l + (r - l) / 2;
    long long ans = inversions(a, buf, l, m)
        + inversions(a, buf, m, r);
    int i = l, j = m, k = l;
    while (i < m && j < r) {
        if (a[i] <= a[j]) buf[k++] = a[i++];
        else { ans += m - i; buf[k++] = a[j++]; }
    }
    while (i < m) buf[k++] = a[i++];
    while (j < r) buf[k++] = a[j++];
    copy(buf.begin() + l, buf.begin() + r, a.begin() + l);
    return ans;
}
```

8.02 贪心与证明 ***

思维关联 T2 排除候选 · T8 调整处理顺序

保留至少一个最优解的同时承诺当前选择

解决什么 / 识别信号 用可证明的局部选择构造全局最优；区间调度、交换不会变差、每步安全边

核心思想 先明确目标，再寻找交换论证、领先论证或割性质。最早结束最大化兼容区间数；单机无释放时间最小最大延迟可用 EDD **

复杂度 / 不变量 通常排序 $O(n \log n)$ +扫描 $O(n)$ ，依题而定。不变量：当前部分解始终可扩展成某个最优解，或在已证明指标上不落后

常见坑 / 替代方案 按最短长度/最高收益直觉选取不自动正确；EDD 与 EDF 的任务、释放时间、抢占前提不同。关联：无法证明时构造小规模反例，比较 DP/穷举；二分答案中的 check 也要独立证明

8.03 回溯、去重与状态恢复 ***

思维关联 T2 排除候选 · T3 合并等价状态 · T8 调整处理顺序

约束剪枝、对称去重与可恢复的分支顺序

解决什么 / 识别信号 枚举所有合法解或寻找一个可行解；子集、组合、排列、分割、棋盘

核心思想 选择→递归→撤销；组合用 start 限制顺序，排列用 used；排序后同一层跳过重复选择，而不同层可能允许相同值

复杂度 / 不变量 子集输出 $O(2^n)$ ，排列输出 $O(n!)$ ；递归辅助空间通常 $O(n)$ 。不变量：path 仅代表当前分支，退出一层后状态完整恢复

常见坑 / 替代方案 用全局去重误删合法重复使用；结果保存引用导致答案被后续修改；漏恢复棋盘。关联：只求计数/最优且有重叠子问题时用 DP；剪枝见第 12 章

8.04 DP 六问与状态表示 ***

思维关联 T3 合并等价状态 · T4 复用计算 · T5 分解与组合

等价未来用同一状态，组合并复用状态值

解决什么 / 识别信号 复用重叠子问题，避免重复搜索；最优值、方案数、可行性，当前决策只依赖有限历史

核心思想 六问：状态含义、选择、转移、初值、计算顺序、答案位置。先写完整状态，再证明哪些维度可压缩；Memoization 是按需计算同一依赖图

复杂度 / 不变量 时间=可达状态数×每状态转移成本；空间=保存的状态数+递归栈。不变量：所有被读取的状态已计算或可递归求得，状态足以决定未来

常见坑 / 替代方案 把路径相关信息漏掉；未访问与结果 0 混淆；环形依赖不能直接记忆化递归。关联：DAG DP 是图上的同一思想；记忆化与迭代 DP 按可达性和栈风险选

8.05 线性、网格与状态机 DP ***

思维关联 T3 合并等价状态 · T4 复用计算 · T8 调整处理顺序

压缩相关历史，并按依赖计算

解决什么 / 识别信号 处理按位置推进的选择；打家劫舍、路径计数、股票持仓、最大子数组

核心思想 线性 DP 保留前几项；网格按依赖方向填；状态机显式记录持仓/冷冻期/交易次数。Kadane 记录“必须以当前位置结束”的最优和

复杂度 / 不变量 常见 $O(n)$ 或 $O(\text{rows} \cdot \text{cols})$ ；空间可滚动压缩。不变量：压缩前后读取的是正确的上一层或当前层版本

常见坑 / 替代方案 全负数组的最大非空子数组不能初始化为 0；障碍起点；非法状态需 -INF 而非 0。关联：只需前后组合也可能用 Prefix/Suffix，避免过度 DP

8.06 0/1、完全与多重背包 ***

思维关联 T3 合并等价状态 · T4 复用计算 · T8 调整处理顺序

容量状态合并历史，循环方向限定可用次数

解决什么 / 识别信号 在容量或总和约束下选择物品；取或不取、可重复选、分割等和、零钱

核心思想 0/1 背包容量倒序以读取上一物品层；完全背包容量正序允许读到本层；多重背包 ** 可二进制拆分成 0/1 物品

复杂度 / 不变量 0/1 和完全背包 $O(nC)/O(C)$ ；多重拆分 $O(C \cdot \Sigma \log(\text{count} + 1))$ 。不变量：容量遍历方向决定每个物品是否能在本轮重复使用

常见坑 / 替代方案 恰好装满时不可达值不能设 0；价值最大化与方法计数初始化不同；容量可能太大。关联：价值维度 DP、bitset 子集和 ** 或 Meet-in-the-middle **

C++ 模板 · 0/1 Knapsack

求容量不超过 C 的最大价值，允许不选；w 与 value 等长， $w[i] > 0$ ， $C \geq 0$ ，结果可表示。完全背包把容量循环改为从 $w[i]$ 到 C 正序。

```
long long knapsack01(const vector<int>& w,
                    const vector<int>& value, int C) {
    vector<long long> dp(C + 1, 0);
    for (int i = 0; i < (int)w.size(); ++i)
        for (int c = C; c >= w[i]; --c)
            dp[c] = max(dp[c], dp[c - w[i]] + value[i]);
    return dp[C];
}
```

零钱计数 ***：dp[0]=1。先枚举硬币、再正序枚举金额，统计不考虑顺序的组合；先枚举金额、再枚举未枚硬币，统计考虑顺序的序列。循环顺序改变的是问题含义，不只是速度。

8.07 LIS: DP 与 tails ***

思维关联 T2 排除候选 · T6 充分摘要 · T4 复用计算

同长度较小尾值支配较大尾值，形成摘要

解决什么 / 识别信号 求最长严格递增子序列长度；保持相对顺序，可跳过元素

核心思想 $O(n^2)$ DP 枚举前驱； $O(n \log n)$ tails[len-1] 存该长度所有候选的最小尾值，用 lower_bound 替换。非递减子序列改 upper_bound

复杂度 / 不变量 长度求解 $O(n \log n)/O(n)$ ，恢复路径额外保存前驱与尾索引。不变量：tails 单调，每个位置是对应长度的最小可实现末尾

常见坑 / 替代方案 tails 本身一般不是原数组中的同一条合法子序列；严格与非严格混用。关联：二维偏序先按第一维排序，同值第二维反向，再 LIS；计数版本需更丰富状态

C++ 模板 · Strict LIS Length

返回长度；空数组为 0。若要求非递减，使用 upper_bound。

```
int lisLength(const vector<int>& a) {
    vector<int> tails;
    for (int x : a) {
        auto it = lower_bound(tails.begin(), tails.end(), x);
        if (it == tails.end()) tails.push_back(x);
        else *it = x;
    }
    return (int)tails.size();
}
```

8.08 字符串 DP: LCS、编辑距离与回文 ***

思维关联 T3 合并等价状态 · T5 分解与组合 · T4 复用计算

前缀或区间定义状态，组合相邻子问题

解决什么 / 识别信号 对齐两个序列或处理成对边界；插删改、公共子序列、回文子序列

核心思想 LCS 相等取左上+1，否则取上/左最大；编辑距离取删除/插入/替换成本最小；回文 DP 从短区间向长区间扩展

复杂度 / 不变量 双串 $O(nm)$ ，滚动空间 $O(\min(n,m))$ ；区间回文常 $O(n^2)/O(n^2)$ 。不变量：dp[i][j] 精确定义为前 i 与前 j 个字符或明确区间的答案

常见坑 / 替代方案 子序列不要求连续，子串要求连续；滚动数组左上值须先保存；边界空串很重要。关联：连续匹配用 KMP/Z；回文子串可中心扩展或 Manacher

8.09 区间 DP 与状态压缩 DP **

思维关联 T5 分解与组合 · T3 合并等价状态 · T4 复用计算

用切分或集合描述依赖并缓存结果

解决什么 / 识别信号 描述切分顺序或集合进度；戳气球、矩阵链； $n \approx 20$ 、访问集合、旅行商

核心思想 区间 DP 枚举最后一步/分割点，按长度递增；Bitmask DP 用 mask 表示已选集合，必要时再加末点 last

复杂度 / 不变量 典型区间 DP $O(n^3)/O(n^2)$ ；TSP DP $O(n^2 \cdot 2^n)/O(n \cdot 2^n)$ 。不变量：转移只依赖更短区间或更小集合，状态没有遗漏影响后续的条件

常见坑 / 替代方案 枚举“第一步”造成左右仍耦合；仅 mask 不足以表示终点；子集枚举总成本可能 3^n 。关联：Digit DP、凸包优化、分治优化列入第 13 章 *

区间 DP 推导：矩阵链相乘 **

矩阵 $A[i]$ 的形状为 $p[i] \times p[i+1]$ 。定义 $dp[i][j]$ 为半开区间 $A[i..j)$ 的最少标量乘法次数。一个矩阵不用乘， $dp[i][i+1]=0$ 。若最后一次乘法在 k 处分成左右两块，新增成本为 $p[i] \cdot p[k] \cdot p[j]$ 。

$dp[i][j] = \min(dp[i][k] + dp[k][j] + p[i] \cdot p[k] \cdot p[j])$ ，其中 $i < k < j$ 。外层按区间长度从 2 到 n ，内层枚举左端和分割点。时间 $O(n^3)$ 、空间 $O(n^2)$ 。三个维度相乘要在足够宽的类型中计算。

例： 10×30 、 30×5 、 5×60 。先乘前两个需 $1500 + 3000 = 4500$ ；先乘后两个需 $9000 + 18000 = 27000$ 。局部矩阵大小不能单独决定最优括号化，区间状态保留所有可能分割。

Bitmask DP 推导：访问所有点的最短回路 **

定义 $dp[mask][v]$ ：从固定起点 s 出发，恰好访问 $mask$ 中的点并停在 v 的最小代价。初始化 $dp[1 < s][s] = 0$ ，其余不可达。枚举 u 不在 $mask$ ，沿有效边 $v \rightarrow u$ 更新 $dp[mask|(1 < u)][u]$ 。最终再加 $v \rightarrow s$ 的边，取最小值；若只求路径则不加回边。

这里 $mask$ 无法替代 v ，因为从不同末点出发的下一步成本不同。 $n=20$ 时， $n \cdot 2^n$ 个 8 字节值就约 160 MiB，尚未计容器开销。状态压缩减少维度表示成本，不会消除指数规模。

09 特殊面试技巧

整理高频但不属于单一设计范式的模式：位操作、缺失与重复、原地索引、候选消除、离线处理及随机化。

9.01 Bit 操作与无符号语义 ***

思维关联 T1 转换与建模 · T6 充分摘要 · T7 贡献与抵消
用位表示有限状态或利用 XOR 抵消

解决什么 / 识别信号 紧凑状态、快速集合操作与奇偶抵消；开关、权限、XOR、计数、掩码

核心思想 使用无符号类型；测试/设置/清除/翻转用位运算， $x \& (x-1)$ 清最低 1；lowbit 用 $x \& (\sim x + 1)$ 表达无符号版本

复杂度 / 不变量 机器字内通常 $O(1)$ ；枚举 set bits 为 $O(\text{popcount}(x))$ 。不变量：每个 bit 的语义固定，所有移位数都小于类型位宽

常见坑 / 替代方案 $1 < i < 31$ 可能触及符号位；移位达到位宽是未定义行为；比较与位运算优先级需括号。关联：C++20 `<bit>` 可读性更好；超机器字集合用 `bitset` 或动态位集

操作 ***	写法 (<code>uint64_t x, 0 ≤ i < 64</code>)	边界
测试 / 设置	<code>(x & (1ULL << i)) != 0 / $x \mid= 1ULL << i$</code>	确保常量宽度与类型相配
清除 / 翻转	<code>$x \&= \sim(1ULL << i)$ / $x \wedge= 1ULL << i$</code>	括号保留意图
清最低 1 / 取最低 1	<code>$x \&= x-1$ / $x \& (\sim x + 1)$</code>	$x=0$ 时都得到 0（无符号）
判断 2 的幂	<code>$x \neq 0 \&\& (x \& (x-1)) == 0$</code>	0 不是 2 的幂
C++20 <code><bit></code>	<code>popcount, countr_zero, countl_zero, has_single_bit</code>	标准函数有规定的零输入行为
向上取 2 的幂	<code>std::bit_ceil(x)</code>	结果必须可表示，否则不能调用；不要盲目用于最大值
XOR 交换	优先 <code>std::swap(a,b)</code>	别名会使 XOR 交换出错，且无实际优势

9.02 子集、子掩码与位迭代 **

思维关联 T1 转换与建模 · T8 调整处理顺序
以掩码编码集合，按结构枚举而不漏项

解决什么 / 识别信号 按集合状态枚举所有选择；小 n 子集 DP、枚举已选集合的一部分

核心思想 子集范围 $[0, 2^n)$ ；`sub=(sub-1)&mask` 枚举子掩码；每次 `x&=x-1` 枚举一个 set bit

复杂度 / 不变量 所有子集 $O(2^n)$ ；一个 mask 的子掩码 $O(2^k)$ ，所有 mask 的所有子掩码 $O(3^n)$ 。不变量：sub 始终是 mask 的子集，数值递减直到 0

常见坑 / 替代方案 0 子掩码要显式退出，否则无符号减 1 会循环；`1ULL << n` 要求 $n < 64$ 。关联：Gray Code * 相邻状态只变一位；XOR Basis * 见第 13 章

C++ 模板 · Submask Enumeration

枚举包括空集；单个 mask 的复杂度取决于其 1 的数量，不是数值大小。

```
template<class Visit>
void eachSubmask(uint64_t mask, Visit visit) {
    uint64_t sub = mask;
    for (;;) {
        visit(sub); // includes zero
        if (sub == 0) break;
        sub = (sub - 1) & mask;
    }
}
```

9.03 Missing / Duplicate 决策树 ***

思维关联 T1 转换与建模 · T7 贡献与抵消 · T6 充分摘要

输入承诺决定后继图、抵消或频次表示

解决什么 / 识别信号 按取值范围、频率承诺和可修改性选算法；缺一个数、找重复、唯一数、最小缺失正整数

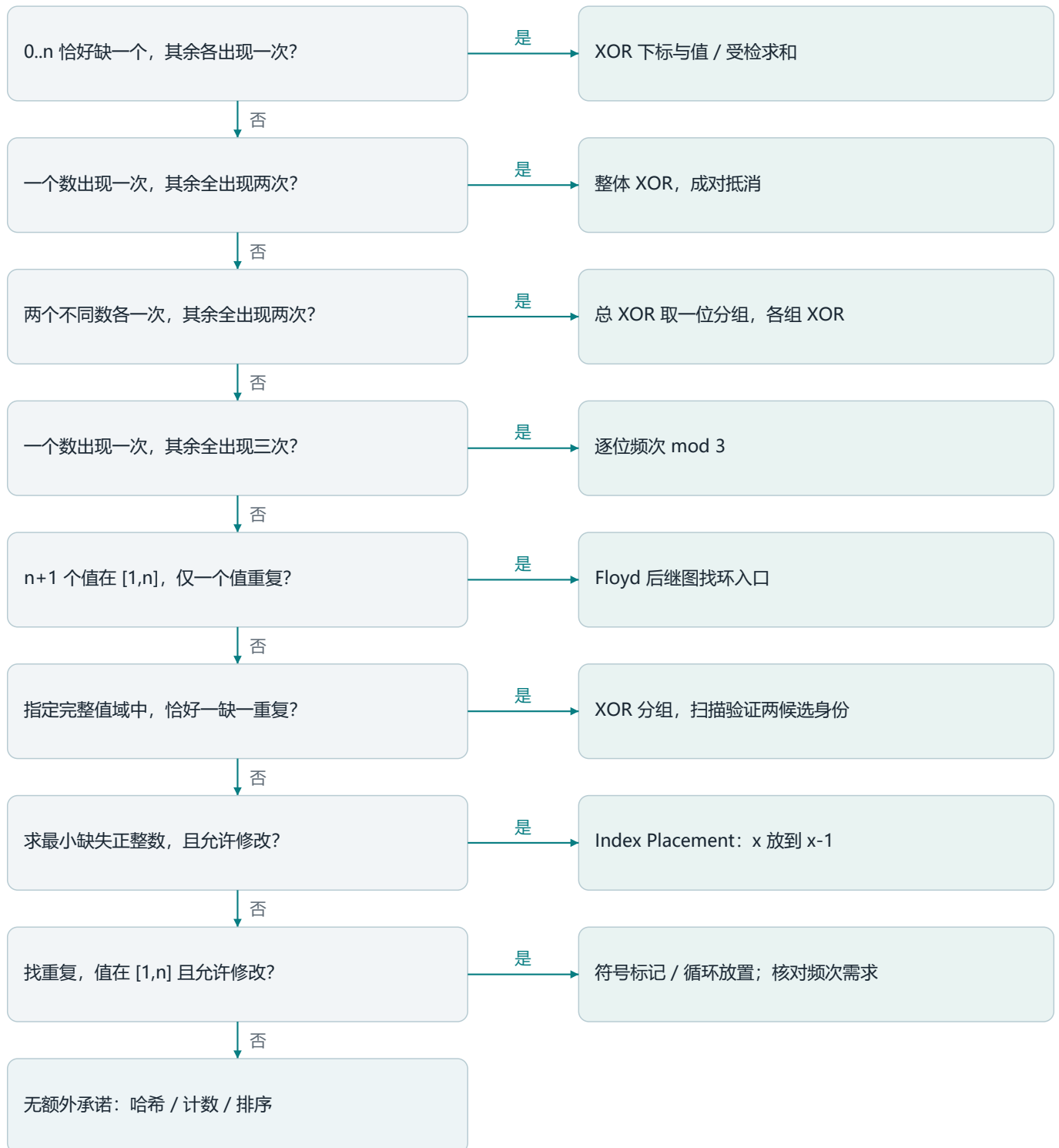
核心思想 先问四件事：值域是否有限；是否保证特定出现次数；能否改数组；要求 $O(1)$ 空间吗。XOR 只保留奇偶信息，不能恢复任意重复分布

复杂度 / 不变量 特定契约下常 $O(n)/O(1)$ ；通用频数 $O(n)$ 期望 $O(n)$ 。不变量：算法消去或定位的信息恰好与题目承诺对应

常见坑 / 替代方案 把 XOR 用于没有频率承诺的任意重复题；Floyd 的输入范围不满足会越界。关联：下面决策表就是逐分支执行的决策树；无承诺时回到排序/哈希

Missing / Duplicate 决策树 ***

从上向下逐项确认完整输入承诺；“是”走右侧方案，“否”继续。普通重复值问题不要默认满足 XOR 或 Floyd 的前提。



决策树的适用契约与解释 ***

先检查的问题	满足时的分支	方法 / 原因
是否恰为 0..n 缺一个，其他各一次？	是 → 单缺失	XOR 所有下标/值/末尾 n；或受检求和
是否一个数一次，其余全部两次？	是 → 单唯一	整体 XOR；成对抵消
是否两个不同数各一次，其余全部两次？	是 → 双唯一	总 XOR 非零，取一位分两组再 XOR
是否一个数一次，其余全部三次？	是 → 模 3 频次	逐位计数取模；负数用固定宽度无符号表示处理
是否 n+1 个值都在 [1,n]，仅一个值重复？	是 → 后继图	Floyd 找环入口；不可改数组且 O(1) 空间时合适
是否恰好一个缺失、一个重复，其余一次？	是 → 成对差异	XOR 分组得到两候选，再扫描分清谁重复
是否找 First Missing Positive，允许改数组？	是 → 下标定位	把合法 x 放到 x-1；首个错位 i 对应 i+1
能改数组且值在 [1,n]？	可选 → 原地标记	符号标记/循环放置；先确认每个值语义
均不满足或需完整频次？	通用分支	哈希/计数数组；或排序后相邻比较

C++ 模板 · Floyd Duplicate

严格要求长度 n+1、n≥1、值域 [1,n]，且只有一个不同的值出现多次。时间 O(n)、额外空间 O(1)，不修改数组。

```
int findDuplicate(const vector<int>& a) {
    int slow = 0, fast = 0;
    do {
        slow = a[slow];
        fast = a[a[fast]];
    } while (slow != fast);
    slow = 0;
    while (slow != fast) {
        slow = a[slow]; fast = a[fast];
    }
    return slow;
}
```

9.04 In-place Indexing 与原地压缩 ***

思维关联 T1 转换与建模 · T6 充分摘要
数组位置承担映射；写指针保留有效前缀

解决什么 / 识别信号 将数组自身当作有限值域映射；1..n、最小缺失正数、原地去重

核心思想 值 x 对应位置 x-1；不断交换直到当前位置正确、越界或遇到相同副本。符号标记依赖原始正数；读写指针用于保序筛选

复杂度 / 不变量 下标定位 O(n)/O(1)：每次有效交换至少固定一个目标；稳定压缩 O(n)/O(1)。不变量：已正确放置的位置不会被无效交换破坏，write 之前全是应保留元素

常见坑 / 替代方案 无重复检查会死循环；先检查范围再索引；频次加 n 编码可能溢出。关联：只读输入用 hash/排序副本；第一缺失正数在 [1,n+1] 由抽屉原理保证

C++ 模板 · First Missing Positive

允许任意 int 值和重复，修改原数组； $n+1$ 必须可表示。不要对负数/大于 n 的值计算索引。

```
int firstMissingPositive(vector<int>& a) {
    int n = (int)a.size();
    for (int i = 0; i < n; ++i) {
        while (a[i] >= 1 && a[i] <= n &&
                a[a[i] - 1] != a[i]) {
            int target = a[i] - 1;
            swap(a[i], a[target]);
        }
    }
    for (int i = 0; i < n; ++i)
        if (a[i] != i + 1) return i + 1;
    return n + 1;
}
```

9.05 多数投票与候选消除 ***

思维关联 T7 贡献与抵消 · T6 充分摘要

抵消产生候选；无存在性承诺时必须验证

解决什么 / 识别信号 在强频率承诺下用常数候选压缩统计；超过 $n/2$ ；超过 $n/3$ **；超过 n/k *

核心思想 Boyer-Moore 抵消不同元素；多数元素若存在不会被全部抵消。 $n/3$ 保留最多两个候选； n/k 保留至多 $k-1$ 个候选

复杂度 / 不变量 固定 k 时 $O(nk)$ 的直接版本、 $O(k)$ 空间；多数单候选 $O(n)/O(1)$ 。不变量：计数反映抵消后尚存候选的净余额，不是实际总频率

常见坑 / 替代方案 题目未保证多数存在时必须第二遍验证；候选相同要先匹配再填空位。关联：一般 Top K 频率用 hash+heap；多数投票不能直接输出精确频次

9.06 Prefix / Suffix、排列与压缩 ***

思维关联 T5 分解与组合 · T8 调整处理顺序 · T1 转换与建模

前后分解；字典序推进；值转秩

解决什么 / 识别信号 把排除一个位置或大值域顺序关系分解；Product Except Self、删除一个后 GCD、下个排列

核心思想 前后缀独立预处理再合并；Next Permutation 找从右起首个上升位置，与右侧最小较大值交换并反转后缀；坐标压缩 ** 排序去重再取秩

复杂度 / 不变量 前后缀 $O(n)$ ；下个排列 $O(n)/O(1)$ ；压缩 $O(n \log n)/O(n)$ 。不变量：前缀和后缀覆盖被排除位置两侧；排列后缀重排为最小可行字典序

常见坑 / 替代方案 Product 不用除法才能自然处理零；压缩保序不保距离；重复排列排名需修正组合计数。关联：只求下个排列用 `std::next_permutation`；第 k 排列/排名 ** 用阶乘与可用元素计数

9.07 Offline Processing: 重排查询 **

思维关联 T8 调整处理顺序 · T4 复用计算

重排后共享逐步激活的工作

解决什么 / 识别信号 用一次单调扫描共享多个查询的工作；查询提前全部给出，不依赖上一个回答

核心思想 保存原 id，按阈值/端点重排查询；数据也按同一关键字排序，只增加活跃数据。典型：查询边权 $\leq T$ 的连通性，按 T 递增把边加入 DSU

复杂度 / 不变量 排序 $O((n+Q)\log(n+Q))$ 加结构操作；空间 $O(n+Q)$ 。不变量：回答阈值 T 时，结构恰好含所有允许的数据，不多不少

常见坑 / 替代方案 严格小于与小于等于决定同值处理顺序；忘记恢复原查询顺序；交互查询未必可离线。关联：删除边可逆序变加边；Mo / CDQ / 离线 LCA 属高阶扩展，见第 13 章

离线阈值连通示例 **

边 $(0,1,2)$ 、 $(1,2,5)$ ，查询 $(0,2,T=4)$ 与 $(0,2,T=5)$ 。按 T 升序处理：先仅合并权 2 的边，答案 false；再加入权 5 的边，答案 true。这里“边权 $\leq T$ ”是激活条件；若改成“边权 $<T$ ”， $T=5$ 的结果仍是 false。

9.08 DSU 扩展：元数据、动态点与势能 **

思维关联 T6 充分摘要 · T1 转换与建模 · T4 复用计算

维护分量摘要、相对关系与根路径信息

解决什么 / 识别信号 在集合合并时维护更多信息；分量大小/和/边数；新 id；相对差与奇偶约束

核心思想 根上合并元数据 **；动态节点 ** 用 id 映射与新根初始化。Weighted DSU * 维护 $\text{delta}[x] = \text{value}[x] - \text{value}[\text{parent}[x]]$ ；Parity DSU * 把差换成 XOR [R3]

复杂度 / 不变量 普通扩展仍摊还 $O(\alpha(n))$ ；动态 id 哈希平均 $O(1)$ ；空间 $O(n)$ 。不变量：find 后 $\text{delta}[x]$ 变为 x 到根的总差；合并方向改变时公式符号也改变

常见坑 / 替代方案 压缩路径时忘记累计旧父的势能；同根需验证约束一致，而不是再次合并。关联：回滚 DSU 不做路径压缩；不能把普通 DSU 的复杂度直接套到所有扩展

Weighted DSU * 推导：find 后令 $\text{dx} = \text{value}[x] - \text{value}[\text{rx}]$ ， $\text{dy} = \text{value}[y] - \text{value}[\text{ry}]$ 。约束 $\text{value}[y] - \text{value}[x] = w$ 。若把 ry 接到 rx ，则 $\text{delta}[\text{ry}] = w + \text{dx} - \text{dy}$ ；若反向挂接，则符号相反。同根时检查 $\text{dy} - \text{dx} = w$ 。Parity 约束 $\text{value}[x] \text{ XOR } \text{value}[y] = b$ ，则根间关系为 $\text{parity}[x] \text{ XOR } \text{parity}[y] \text{ XOR } b$ 。

9.09 随机化与抽样 **

思维关联 T8 调整处理顺序 · T6 充分摘要

随机顺序或选择控制期望成本，样本保留分布性质

解决什么 / 识别信号 避免固定坏输入或从未知长度流均匀抽样；洗牌、随机 pivot、流中抽 K 个

核心思想 Fisher-Yates 第 i 步在 $[0, i]$ 均匀选 j 交换；Reservoir 对第 t 个元素以 K/t 概率纳入，并均匀替换一个槽；随机 pivot 把选择的复杂度结论改为期望

复杂度 / 不变量 洗牌 $O(n)/O(1)$ ；水塘 $O(n)/O(K)$ ；Quickselect 期望 $O(n)$ 。不变量：洗牌的各排列等概率；处理 t 个元素后每个元素入样概率 K/t

常见坑 / 替代方案 $\text{rand}()\%m$ 可能模偏差；每次调用重新种子会制造相关性；随机算法仍有最坏情况。关联：使用均匀整数分布；测试固定种子便于复现；随机 hash * 不等于安全哈希

10 字符串算法

先分清字符语义与问题对象：连续匹配、前缀、子序列、回文、多模式和后缀索引需要不同工具。

10.01 扫描、频数与字符边界 ***

思维关联 T1 转换与建模 · T6 充分摘要

字符定义决定状态表示与计数摘要

解决什么 / 识别信号 处理基础匹配、规范化、回文和异位词；固定字母表、双指针、字符计数

核心思想 字母表很小时用数组；字节序列与 Unicode 字符/字素是不同概念。先确认题目是否只含 ASCII 或小写字母

复杂度 / 不变量 扫描 $O(n)$ ，空间 $O(\sigma)$ ；排序异位词 $O(n \log n)$ 。不变量：每个计数槽与题意字符一一对应

常见坑 / 替代方案 char 可能有符号，作为数组索引先转 unsigned char；tolower 等也需合法字符参数；UTF-8 不能按字节翻转当作字符翻转。关联：复杂 Unicode 处理依赖库；面试先确认输入契约，避免无意改变语义

10.02 KMP 与 Z Algorithm **

思维关联 T4 复用计算 · T2 排除候选

复用匹配关系，排除已经证明不匹配的长度

解决什么 / 识别信号 线性单模式匹配和前缀相似度；重复前缀、寻找所有匹配位置、周期

核心思想 KMP $\pi[i]$ 是前缀中最长真前后缀长度，失败后退到 $\pi[j-1]$ ；Z[i] 是从 i 起与全串前缀的最长公共长度，复用最右匹配区间

复杂度 / 不变量 预处理 $O(m)$ ，匹配 $O(n+m)$ ，空间 $O(m)$ ；Z 整串 $O(n)$ 。不变量：KMP 已匹配部分始终等于模式某前缀；Z 的 $[l,r)$ 是已知最右匹配区间

常见坑 / 替代方案 真前后缀不能等于整个串；空模式语义需约定；拼接分隔符不能与内容冲突。关联：简单查一次可用标准查找；多模式用 AC；字符串哈希有碰撞 [R4]

C++ 模板 · KMP Prefix Function

支持重叠匹配；空模式定义为匹配所有 $n+1$ 个边界。按字节比较，字符串长度须可用 `int` 索引。

```
vector<int> prefixFunction(const string& s) {
    vector<int> pi(s.size(), 0);
    for (int i = 1; i < (int)s.size(); ++i) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) ++j;
        pi[i] = j;
    }
    return pi;
}

vector<int> kmpFind(const string& text, const string& pat) {
    vector<int> ans;
    if (pat.empty()) {
        for (int i = 0; i <= (int)text.size(); ++i)
            ans.push_back(i);
        return ans;
    }
    auto pi = prefixFunction(pat); int j = 0;
    for (int i = 0; i < (int)text.size(); ++i) {
        while (j > 0 && text[i] != pat[j]) j = pi[j - 1];
        if (text[i] == pat[j]) ++j;
        if (j == (int)pat.size()) {
            ans.push_back(i - j + 1); j = pi[j - 1];
        }
    }
    return ans;
}
```

10.03 Rolling Hash / Rabin-Karp **

思维关联 [T4 复用计算](#) · [T6 充分摘要](#)

复用可组合指纹；相等判断保留碰撞边界

解决什么 / 识别信号 快速比较子串或寻找候选匹配；多次子串相等、最长重复片段、二分长度

核心思想 多项式前缀哈希加幂表，区间哈希由两前缀相减得到；Rabin-Karp 滑动更新候选哈希

复杂度 / 不变量 预处理 $O(n)$ ，单次哈希 $O(1)$ ，空间 $O(n)$ ；逐字符确认最坏成本可能更高。不变量：同一编码、基数和模数下，相同串哈希相同

常见坑 / 替代方案 哈希相同不保证字符串相同；双模只是降低碰撞风险；模乘可能溢出。关联：需要确定性证明用 KMP/Z/后缀结构；可对候选做精确确认

10.04 回文：中心扩展与 Manacher **

思维关联 [T4 复用计算](#) · [T5 分解与组合](#)

按中心分解，镜像半径复用已知匹配

解决什么 / 识别信号 求回文子串数量或最长回文子串；对称、连续回文、奇偶中心

核心思想 中心扩展 ** 同时考虑字符中心和间隙中心；Manacher * 利用镜像半径和当前最右回文边界避免重复扩展

复杂度 / 不变量 中心扩展 $O(n^2)/O(1)$ ；Manacher $O(n)/O(n)$ 。不变量：已记录半径对应真实回文；镜像复用只能当前边界内保证

常见坑 / 替代方案 偶数长度中心遗漏；变换串哨兵冲突；回文子序列需要 DP。关联：只判断一个完整字符串用双指针 ***；回文奇偶可行性用 Prefix Mask **

10.05 多模式与后缀结构 *

思维关联 T1 转换与建模 · T3 合并等价状态 · T4 复用计算 · T8 调整处理顺序
前缀复用、后缀排序或自动机等价状态

解决什么 / 识别信号 在大文本中搜索多词或建立子串索引；词典全匹配、重复子串、不同子串数

核心思想 Aho-Corasick 用 Trie+失败指针；Suffix Array 排序所有后缀并构造 LCP；Suffix Automaton 以等价右上下文压缩子串

复杂度 / 不变量 AC 固定字母表稠密转移 $O(\text{总模式长} \cdot \sigma + \text{文本长} + \text{输出数})$ ；倍增 SA 常 $O(n \log n)$ （每轮线性排序）；SAM 固定字母表 $O(n)$ 。不变量：失败指针指向最长可用后缀；SA 按后缀字典序；SAM 状态长度区间表示一族子串

常见坑 / 替代方案 AC 输出数量可能巨大；比较排序实现的倍增 SA 可 $O(n \log^2 n)$ ；SAM 克隆状态容易写错。关联：单模式优先 KMP；纯前缀查询用 Trie；这些为岗位相关时再深入

字符串结构的目标对照 ** / *

真实目标	优先模型	保留的信息
词是否存在 / 哪些词以此开头	Trie **	共享的前缀路径
一个模式在文本中所有出现位置	KMP / Z **	前后缀重叠与匹配进度
一批模式同时出现的位置	Aho-Corasick *	Trie 状态与失败链接
后缀排序、子串字典序、重复片段	Suffix Array + LCP *	后缀秩及相邻公共前缀
不同子串、子串出现次数等自动机问题	Suffix Automaton *	子串的等价结束位置集合
快速子串相等候选过滤	Rolling Hash **	可组合指纹，但存在碰撞

后缀数组的小例子 *

banana 的后缀按字典序排列为 a、ana、anana、banana、na、nana，对应起点 [5,3,1,0,4,2]。相邻 LCP 为 [1,3,0,0,2]。最长重复子串长度等于最大 LCP，即 3；不同非空子串数为 $n(n+1)/2 - \sum \text{LCP} = 21 - 6 = 15$ 。

这类结构的价值来自大量相关查询共享预处理。若只找一个模式一次，建立后缀数组通常增加实现负担；先用更简单、前提清楚的匹配算法。

11 数学与数论

数学技巧必须连同适用条件一起记忆；位宽、模数和计数规模经常比公式本身更容易出错。

11.01 GCD、LCM 与抽屉原理 ***

思维关联 T1 转换与建模 · T2 排除候选

取余保持公因数集合；容量论证排除某些分布

解决什么 / 识别信号 整除、公因数、周期与存在性证明；约分、周期同步、值域小于元素数

核心思想 欧几里得 $\gcd(a,b)=\gcd(b,a\%b)$ ；lcm 先除 gcd 再乘。抽屉原理将“槽位不够”转成必有重复或缺失的结论

复杂度 / 不变量 gcd $O(\log \min(a,b))$ (正输入)，LCM 同阶。不变量：公因数集合在取余后不变

常见坑 / 替代方案 零和负数规范化；lcm 仍可能溢出；抽屉原理证明存在，不直接给出算法。关联：扩展 gcd * 求 $ax+by=\gcd$ ；Missing/Duplicate 利用受限值域

11.02 快速幂与模运算 ***

思维关联 T5 分解与组合 · T4 复用计算

指数二进制分解，复用已算出的幂

解决什么 / 识别信号 计算大指数幂和重复变换； a^e 、模幂、计数取模

核心思想 按指数二进制分解，累计结果并反复平方。乘法取模前必须确保中间乘积可表示

复杂度 / 不变量 $O(\log e)$ 次乘法、 $O(1)$ 空间。不变量： $\text{result} \times \text{base}^{\text{remaining}}$ 与原目标在模意义下相同

常见坑 / 替代方案 负余数归一化；不能把模除法写成普通整数除法；模数 1 和指数 0 要有一致定义。关联：大模数可用更宽乘积或专用模乘；矩阵快速幂 * 扩展到线性递推

C++ 模板 · Modular Power

契约： $1 \leq \text{mod} \leq 3,000,000,000$ ，使 $(\text{mod}-1)^2$ 在 long long 中可表示；e 为非负指数。更大模数不能直接复制乘法。

```
long long modPow(long long a, uint64_t e, long long mod) {
    a %= mod; if (a < 0) a += mod;
    long long result = 1 % mod;
    while (e != 0) {
        if (e & 1) result = result * a % mod;
        a = a * a % mod;
        e >>= 1;
    }
    return result;
}
```

11.03 筛法与质因数分解 **

思维关联 T2 排除候选 · T4 复用计算

共享倍数标记，或排除不可能的因子范围

解决什么 / 识别信号 批量判素、枚举质因子；大量数论查询、质因子贡献

核心思想 埃氏筛从 p^2 开始标记；单个正整数试除到 $d \leq x/d$ ，每除尽一个因子再继续；最小质因子表可加速多次分解

复杂度 / 不变量 埃氏筛 $O(U \log \log U)/O(U)$ ；单次试除最坏 $O(\sqrt{x})$ 。不变量：筛到 p 时，更小质因子的合数已被覆盖

常见坑 / 替代方案 1 不是质数； $p \cdot p$ 和 $d \cdot d$ 可能溢出；缩小 x 后上界也可缩小。关联：线性筛 * $O(U)$ ；大整数素性测试/分解按岗位另学

11.04 组合计数与模逆 **

思维关联 T7 贡献与抵消 · T5 分解与组合 · T4 复用计算

唯一计数分解、公共阶乘预处理与代数转换

解决什么 / 识别信号 组合数、排列数、重复选择和计数取模；选 K 个、排列排名、路径数

核心思想 Pascal $C(n,k)=C(n-1,k-1)+C(n-1,k)$ ；阶乘法用于重复查询。模逆 * 仅当 $\gcd(a,m)=1$ 存在；素数模 p 下 $a^{(p-2)}$ 要求 a 不为 0 模 p

复杂度 / 不变量 Pascal $O(nk)$ ；阶乘预处理 $O(n)$ ，逆阶乘后查询 $O(1)$ 。不变量：每一步的对象有唯一计数归属，避免重复或漏计

常见坑 / 替代方案 $n \geq p$ 时阶乘可能为 0，不可直接取逆；组合数增长很快；计数 DP 忘记取模。关联：大组合数可用精确整数；Lucas/CRT 属高阶；重复元素排列需除去重复阶乘

11.05 扩展数论与矩阵递推 *

思维关联 T1 转换与建模 · T5 分解与组合 · T4 复用计算

把约束或递推转成可合并的代数对象

解决什么 / 识别信号 解线性同余、多模条件、快速递推； $ax \equiv b \pmod{m}$ 、周期组合、巨大步数

核心思想 扩展 gcd 求系数；CRT 合并同余并检查相容性；矩阵幂把有限线性状态推进 e 次

复杂度 / 不变量 扩展 gcd $O(\log m)$ ； $d \times d$ 朴素矩阵幂 $O(d^3 \log e)/O(d^2)$ 。不变量：合并后的同余同时满足已处理条件；状态向量与矩阵乘法顺序一致

常见坑 / 替代方案 非互素 CRT 需差值被 gcd 整除；矩阵维度和初始下标易错；模乘溢出。关联：小阶递推可用直接公式或更专门方法；不必为简单 Fibonacci 引入大矩阵

11.06 计算几何基础 *

思维关联 T1 转换与建模 · T2 排除候选 · T8 调整处理顺序

几何关系变谓词；排序后剔除凸包内部候选

解决什么 / 识别信号 几何方向、相交和外边界；点集、线段、矩形覆盖

核心思想 叉积判断转向；线段相交结合方向与共线包围盒；凸包用排序+单调链，扫描线维护几何活跃集合

复杂度 / 不变量 线段判断 $O(1)$ ；凸包 $O(n \log n)/O(n)$ 。不变量：凸包链始终保持指定转向；几何谓词在数值精度下保持一致

常见坑 / 替代方案 叉积需先扩宽再减乘；边界共线是否保留需约定；浮点 epsilon 不能通吃所有尺度。关联：整数坐标尽量精确计算；欧氏距离比较可用平方但仍需防溢出

12 搜索空间优化

先定义状态和正确性，再减少要访问的状态。剪枝或启发式若没有依据，会把正确答案一起删掉。

12.01 剪枝、记忆化与状态编码 ***

思维关联 T2 排除候选 · T3 合并等价状态 · T4 复用计算

分别减少分支、重复状态和重复计算

解决什么 / 识别信号 减少重复探索或不可能成功的分支；排列组合搜索、重复局面、容量上界

核心思想 可行性剪枝删不合法状态；最优性剪枝用严格可信的界；记忆化按未来行为等价的状态存结果。状态压缩不是简单“少放几个字段”

复杂度 / 不变量 从搜索树节点数降到可达不同状态数；编码/查表成本须计入。不变量：被合并状态的后续可选动作与结果相同；被剪分支不可能改进答案

常见坑 / 替代方案 忽略剩余资源或 visited 集合导致缓存串状态；过强剪枝丢解。关联：Dominance/Pareto ** 可在资源与收益方向一致时删劣状态

12.02 Dominance / Pareto 状态剪枝 **

思维关联 T2 排除候选

以未来延续可比较为前提删除劣状态

解决什么 / 识别信号 淘汰在所有未来选择下都不如另一状态的候选；相同位置、多维资源、成本与剩余预算

核心思想 若状态 A 成本不高于 B、可用资源不少于 B，且二者未来动作集合可比较，则 B 被支配。保留互不支配的前沿

复杂度 / 不变量 前沿可能仍指数级；维护成本依维数和数据结构。不变量：被删除状态的任一续接解可由保留状态达到且不差

常见坑 / 替代方案 仅比较当前得分不够；历史限制或负资源收益会破坏支配关系。关联：单维最优值是最简单的支配压缩；条件不明时保留完整状态

12.03 Bidirectional BFS **

思维关联 T5 分解与组合 · T8 调整处理顺序

从两端组织搜索并用层序保证会合最优性

解决什么 / 识别信号 减少已知起点终点之间的无权搜索范围；单对最短操作、可逆或可生成反向边

核心思想 分别从起点和终点按层扩展，通常扩展较小的当前层；有向图终点侧用反图。维护两边距离，并用完整层与已发现最优会合距离构造停止条件

复杂度 / 不变量 理想树状状态空间从 $O(b^d)$ 降到约 $O(b^{(d/2)})$ ；最坏仍 $O(V+E)$ 。不变量：两侧各自 BFS 层序正确，候选答案等于两侧路径长度之和

常见坑 / 替代方案 任意异步逐点扩展后遇到第一个交点就停可能缺少最优性证明；不可逆状态不能凭空反搜。关联：若难以严谨实现终止条件，先用单源 BFS；多目标最近源用多源 BFS

12.04 Meet-in-the-middle **

思维关联 T5 分解与组合 · T8 调整处理顺序
枚举两半，排序或哈希后高效匹配

解决什么 / 识别信号 把指数枚举拆为两半再合并； $n \approx 40$ 的子集和、难以按和值做 DP

核心思想 两半独立枚举所有状态，排序/哈希匹配目标；最大不超过容量可对一半排序再二分

复杂度 / 不变量 约 $O(2^{n/2} \cdot \text{poly}(n))$ 时间、 $O(2^{n/2})$ 空间。不变量：合并的一对半边状态唯一对应一个完整选择

常见坑 / 替代方案 两半组合仍做全笛卡尔积会退回 2^n ；重复和值的计数不能随意去重。关联： $n \approx 20$ 用位掩码；值域小用背包；存在支配关系可压缩半边状态

12.05 迭代加深、Branch and Bound 与 A* *

思维关联 T8 调整处理顺序 · T2 排除候选
调整探索次序，使用可信下界剪枝或定案

解决什么 / 识别信号 在内存、搜索顺序和最优性之间折中；深度未知、最优方案、可构造下界的路径

核心思想 迭代加深反复做深度限制 DFS；分支定界维护当前最好解并用界淘汰分支；A* 按 $g+h$ 优先扩展

复杂度 / 不变量 复杂度取决于搜索空间和启发式，最坏仍可能指数级；A* 常需大量内存。不变量：h 不高估剩余代价可保证树搜索可采纳；图搜索若不重新打开状态，通常需要一致启发式

常见坑 / 替代方案 启发值高估可能丢最优；只存 visited 不允许更优 g 更新；把启发式当正确性证明。关联：h=0 退化 Dijkstra；IDA* 以 f 阈值迭代加深，适合内存受限场景

A* 的两个条件不能混为一谈 *

条件	定义	作用
可采纳	$h(v)$ 不超过 v 到目标的真实最小代价	提供可信下界；还需匹配正确的搜索/重开策略
一致性	对边 $u \rightarrow v$ ， $h(u) \leq w(u,v) + h(v)$ ，且目标 $h=0$	非负边权常用图搜索中可支持节点定案，不必重开
终止时机	目标作为当前有效最小 f 状态被取出	不能在目标第一次入队时就宣称最优

四方向、每步成本 1 的网格中，曼哈顿距离忽略障碍后是下界，也满足一致性。若允许成本 1 的对角移动，从 (0,0) 到 (1,1) 只需一步，但曼哈顿距离为 2，已经高估；此时应更换启发式，例如与移动规则匹配的 Chebyshev 距离。

剪枝自检：删掉的是冗余，还是可能的答案？ ***

找最小代价时，若“当前代价+可信剩余下界 $\geq$ 已有最优值”，可以剪掉只会打平或更差的分支；但若需要枚举所有最优解，等号就可能不能剪。包含负代价时，“当前代价已经太大”本身也不足以剪枝，因为后续可能降低总代价。
优化顺序建议：先写出正确状态图和基准解，再加记忆化，再加有证明的剪枝，最后调整探索顺序。每增加一种剪枝，都用小规模穷举对拍，验证它改变的是工作量而不是答案集合。

13 高级结构、算法与岗位扩展

这一章是按需深入的路线图。除标为 ** 的工程常见结构外，* 条目以理解接口、限制和实现风险为目标。

13.01 Lazy、持久化与树链剖分 *

思维关联 T8 调整处理顺序 · T4 复用计算 · T5 分解与组合

延迟传播、共享版本、路径分解各有作用

解决什么 / 识别信号 复杂范围更新、历史版本和树路径查询；批量赋值/加值；查询旧版本；树上路径

核心思想 Lazy 标记表示尚未下推的区间变换，组合顺序必须明确；持久化线段树只复制访问路径；HLD 将路径拆为 $O(\log n)$ 条重链区间

复杂度 / 不变量 Lazy 常每次 $O(\log n)$ ；持久化每次新建 $O(\log n)$ 节点；HLD+线段树路径查询常 $O(\log^2 n)$ 。不变量：节点聚合已反映本节点标记，孩子可能尚未反映；旧版本节点不可变

常见坑 / 替代方案 赋值与加法不可随意交换；空标记不一定等于数值 0；非交换路径聚合需处理方向。关联：子树问题先用 Euler Tour；单点增量和用 Fenwick，复杂度较低也更易实现

13.02 平衡树、Treap 与 Order Statistics *

思维关联 T6 充分摘要 · T8 调整处理顺序 · T5 分解与组合

维护顺序与子树摘要，局部重组保持效率

解决什么 / 识别信号 动态有序集合、秩和第 k 小；插删交错、区间序列操作、动态排名

核心思想 平衡 BST 保证高度；Treap 以键保持 BST、随机优先级保持堆；加 subtree size 支持秩。隐式 Treap 按位置组织序列

复杂度 / 不变量 平衡树 $O(\log n)$ 最坏；随机 Treap $O(\log n)$ 期望；空间 $O(n)$ 。不变量：旋转或 split/merge 后，顺序、优先级和子树统计都成立

常见坑 / 替代方案 重复键计数遗漏；指针和内存回收复杂；GNU PBDS 不是标准 C++。关联：静态排名用排序+二分；有限值域动态排名用 Fenwick；map 不自带 $O(\log n)$ rank

13.03 离线高级：Mo、CDQ、回滚 DSU *

思维关联 T8 调整处理顺序 · T4 复用计算 · T5 分解与组合

重排、分治与回滚共享可撤销的状态维护

解决什么 / 识别信号 处理可重排范围查询和动态连通的时间维；容易 add/remove 的区间统计；多维偏序；加删边

核心思想 Mo 调整左右端点减少移动；CDQ 按一维分治，在合并时解决跨组关系；线段树分治时间+回滚 DSU 把边活跃期分配到时间区间

复杂度 / 不变量 Mo 常 $(n+Q)\sqrt{n}$ 次移动；CDQ+Fenwick 常 $O(n \log^2 n)$ ；回滚 DSU 按大小合并 find $O(\log n)$ 。不变量：进入时间节点前应用该段永久活跃边，离开时回到快照；回滚栈记录每次变更

常见坑 / 替代方案 回滚版一般禁用路径压缩；Mo 要求统计可逆维护；删除相同边需处理多重出现。关联：只删不加可倒序用普通 DSU；Offline LCA 用 DFS+DSU，避免为简单查询引入复杂框架

13.04 网络流、最小割与匹配 *

思维关联 T1 转换与建模 · T4 复用计算

容量图建模；残量保留修改旧决策的能力

解决什么 / 识别信号 容量约束下的运输、分配和可行性；二分匹配、互斥选择、最小切断代价

核心思想 最大流沿残量网络增广；反向边允许撤回旧决策；Dinic 分层 BFS+阻塞流；最大流值等于最小割容量。匹配可看作单位容量流

复杂度 / 不变量 一般 Dinic $O(V^2E)$ ；简单二分图 DFS 增广 $O(VE)$ ；Hopcroft-Karp $O(E\sqrt{V})$ 。不变量：流量守恒、容量约束与反向残量一致

常见坑 / 替代方案 漏加反向边；反向边索引错误；容量总和溢出；最小费用流还需维护费用条件。关联：二分图可先写匹配专用算法；没有容量约束时常是普通最短路或贪心

13.05 Digit DP、DP 优化与 XOR Basis *

思维关联 T3 合并等价状态 · T2 排除候选 · T6 充分摘要

状态合并、候选支配或线性空间摘要

解决什么 / 识别信号 巨大数值范围计数或优化特定转移；数位限制；转移含线性函数；异或可达集合

核心思想 Digit DP 用 pos/tight/started/业务状态；分治 DP 优化依赖决策单调；凸包技巧维护线性函数最值；XOR Basis 以最高位消元

复杂度 / 不变量 Digit DP=位数×状态×10；分治优化常每层 $O(n \log n)$ ；W 位基插入/求最大 XOR $O(W)$ 。不变量：DP 优化必须保留已证明的最优候选；XOR 基每个主元位唯一

常见坑 / 替代方案 未证明四边形不等式/决策单调就套优化；数位前导零改变业务状态；XOR 基不是有序数集合。关联：先得到正确朴素 DP 再优化；普通最大 XOR 配对用 Bitwise Trie **

13.06 LRU / LFU、Bloom Filter 与一致性哈希 **

思维关联 T6 充分摘要 · T8 调整处理顺序 · T1 转换与建模

需求决定缓存顺序、概率摘要或分片表示

解决什么 / 识别信号 有限缓存淘汰与大集合近似查询；设计缓存、查找热点、分片迁移

核心思想 LRU ** 用 hash+双向链表；LFU * 用键表+频次桶+最小频次。Bloom * 多 hash 设置 bit，允许假阳性；一致性哈希 * 用环和虚拟节点分散迁移

复杂度 / 不变量 LRU 平均 $O(1)/O(\text{capacity})$ ；桶式 LFU 平均 $O(1)$ ；Bloom $O(k)$ 次位操作；环定位 $O(\log v)$ 。不变量：LRU 链表保持使用先后且与索引一致；Bloom 插入后对应位不清零

常见坑 / 替代方案 容量 0；erase 后迭代器；Bloom 不能普通删除且不是精确成员表；一致性哈希不自动解决热点。关联：高命中小表可直接数组/扁平结构；缓存策略必须结合工作负载测试

13.07 低延迟数据布局与并发边界 **

思维关联 T1 转换与建模 · T8 调整处理顺序

布局、处理次序与延迟执行需接受成本模型约束

解决什么 / 识别信号 让正确算法在目标环境中更可预测；持续流处理、热点循环、大量小对象、尾延迟

核心思想 减少分配与指针追踪，预留容量，考虑 SoA/AoS 与批处理；缓存感知结构 **、环形缓冲 **。Lock-free

* 要明确 SPSC/MPSC/MPMC 模型、发布顺序与回收方案

复杂度 / 不变量 渐进复杂度相同也可能有显著常数/尾延迟差异；并发进展保证需单独证明。不变量：对象生命周期、所有权和内存可见性规则始终成立

常见坑 / 替代方案 摊还 $O(1)$ 不代表单次延迟稳定；false sharing；ABA；无锁不等于 wait-free，也不保证更快。

关联：面试先给单线程正确版本，再按已明确的并发与延迟需求扩展；基准测试覆盖真实数据分布

岗位边界：本章给出算法面试所需的性能思路，不把简化代码包装成交易系统组件。生产组件还需分配策略、线程模型、异常策略、内存回收和测量设计的独立审查。

14 模型识别地图

第 02 章提供思维入口，本章提供题面入口。把“题面信号 → 必须确认的条件 → 候选模型 → 排除错误方案”连起来。这是复习入口，不是看到关键词就直接套模板。

30 秒识别流程 ***

步骤	快速问自己	落点
1 / 确定对象	连续区间、子序列、集合、树、图，还是状态空间？	连续性、顺序性和连通性决定第一层分类
2 / 确定任务	存在、计数、最优值、枚举、动态维护？	输出不同，状态与复杂度也不同
3 / 确定限制	n/Q/U 多大？能否修改？查询可否离线？边权是什么？	先排除不可行算法
4 / 寻找可利用结构	单调、前缀可抵消、局部支配、DAG、一个后继？	将题面变成可证明的模型
5 / 写出契约	状态是什么？不变量是什么？哪些输入会失败？	确认后才选代码模板

序列、频数与范围 *** / **

题面信号	先确认	首选与替代	回看
连续子数组的和等于 K	是否含负数？求个数还是长度？	Prefix+Hash ***；非负且有单调约束可窗口	05
恰好 K 种元素	能否求 atMost(K)？	两次计数窗口 ***	05
平衡 0/1 或字符奇偶	状态能否抵消？	Balance *** / Prefix Mask **	05
窗口最大 / 最小	窗口单向滑动？	单调队列 ***；任意删除用树	05
最近更大 / 更小	严格性和方向？	单调栈 ***	05
所有子数组极值之和	每个值负责哪些区间？	单调边界+贡献法 ***	05
第 k 小 / Top K	静态？流式？结果要有序？	nth_element / 堆 / partial_sort ***	04
合并 K 个有序流	每路是否都单调？	K-way heap ***	05
动态中位数	是否需要删除？	双堆 ** / 两个 multiset	05
大量区间查询	是否有更新？聚合是否可逆/幂等？	Prefix *** / BIT ** / Segment Tree **	03
值为 1..n，常数空间	允许修改？频次有什么承诺？	下标定位 *** / Floyd ***	09
只有一个/两个奇数次元素	是否真的满足“两次”承诺？	XOR ***；一般频次用哈希	09
逆序、右侧比它小	静态还是在线？	归并计数 *** / Fenwick **	08
值很大但只关心顺序	还需要原始距离吗？	坐标压缩 **，保存原坐标差	09

树、图与离线 ** / ***

题面信号	先确认	首选与替代	回看
最少操作次数	所有转移是否同代价?	BFS ***; 否则按边权选最短路	07
同时扩散 / 最近源	源是否有多个? 边权统一?	多源 BFS *** / 多源 Dijkstra	07
先修 / 依赖	有向? 可能有环?	拓扑排序+环检测 ***	07
两点是否连通, 不断加边	是否无向? 会删边吗?	DSU ***; 删边考虑离线 **	03 / 09
有向图互相可达	需要强连通分量?	SCC **, 缩点变 DAG	07
每个点只有一个 successor	是否允许没有后继?	Functional Graph ** / Floyd	07
树中一个节点的所有后代	子树而非任意路径?	Euler Tour ** + 区间结构	06
第 k 个祖先 / 跳很多次	最大 k 的位数?	Binary Lifting **	06
树上任意路径聚合	节点值还是边值?	LCA **; 动态路径用 HLD *	06 / 13
最小连接全部节点费用	求总连接成本而非源距离?	MST **	07
查询可重排、按阈值变化	是否依赖前一个答案?	Offline Sorting ** + DSU/BIT	09
删边后连通性	所有操作预知? 只删还是混合?	逆序加边 ** / 回滚 DSU *	13
两类对象分配且各用一次	是否二分结构?	二分匹配 *; 先二染色 **	07 / 13

优化目标与搜索空间 *** / ** / *

题面信号	先确认	首选与替代	回看
最小化最大值 / 最大化最小值	可行性是否单调? check 正确吗?	Binary Search on Answer ***	04
选择顺序影响未来	有限状态能概括历史吗?	DP ***; 无重叠则回溯	08
所有组合 / 排列	输出可能有多大? 是否有重复?	Backtracking ***	08
n≈20 的集合选择	是否还需末点/剩余资源?	Bitmask DP ** / 子集枚举	08
n≈40、和值巨大	能拆为两个独立半边吗?	Meet-in-the-middle **	12
容量小、物品多	可重复使用? 恰好还是不超过?	Knapsack ***	08
巨大整数区间中的计数	限制是否可逐位累积?	Digit DP *	13
反复匹配一个模式	要精确匹配还是候选过滤?	KMP/Z **; Rolling Hash **	10
词典全部匹配	模式有多少? 总长度多大?	Aho-Corasick *	10
低延迟动态统计	更新分布、分配和最坏路径?	先选正确结构, 再测布局/尾延迟 **	13 / 16

五组最容易误判的相似题 ***

相似表述	决定性的差别	正确分流
子数组 / 子序列	是否必须连续	子数组 → 窗口/前缀；子序列 → DP/LIS
最短路 / 最小生成树	单源距离 / 全局连接总成本	BFS/Dijkstra 等 / Kruskal、Prim
区间调度 / 会议室数	选择最多兼容区间 / 同时占用数量	最早结束贪心 / 事件或小根堆
字符串前缀 / 子串 / 子序列	固定起点 / 连续段 / 可跳过	Trie/Z/KMP / KMP/后缀结构 / LCS
可行性单调 / 数据有序	布尔答案域结构 / 数组顺序	二分答案 / 下标边界二分；二者可同时出现

把地图用于真实题面：三次完整选型 *** / **

- 案例 A：数组可含负数，统计和为 K 的连续子数组。对象是连续区间，任务是计数，负数使“和超了就缩左端”失效。改用 $S[j]-S[i]=K$ ，保存历史前缀频次，先查后增；平均 $O(n)$ 时间、 $O(n)$ 空间。若改问最长长度，频次表改为最早位置表。***
- 案例 B：树上 Q 次单点修改与子树求和。对象是子树，操作在线交错，前缀和不能直接支持快速修改。Euler Tour 把子树变 $[tin,tout)$ ，Fenwick 存展平后的节点值；一次预处理 $O(n)$ ，每次操作 $O(\log n)$ ，额外 $O(n)$ 。若改成任意路径求和，还要补充 LCA/HLD 等建模。**
- 案例 C：查询两点能否仅通过权重 $\leq T$ 的边连通。对象是无向连通关系，目标不是最短距离，所有查询可预读。按 T 排序，边按权值逐步激活并 DSU 合并，最后恢复查询 id；排序占 $O((E+Q)\log(E+Q))$ ，合并和查询接近线性。若查询必须依赖上一答案才生成，这个离线顺序未必可用。**

15 复杂度与算法选型速查

这些表用于筛选候选方案。经验规模不能替代对实际操作量、常数、内存与时间限制的估算。

输入规模 → 首轮候选 ***

典型量级	可考虑的量级	必须继续检查
$n \leq 10$	$O(n!)$ 、回溯	$10!$ 已约 360 万；复制完整解还要乘 n
$n \approx 20$	$O(2^n)$ 、 $O(n \cdot 2^n)$	2^{20} 约 105 万； $n^2 \cdot 2^n$ 可能已过大
$n \approx 40$	Meet-in-the-middle	两半各约 2^{20} 状态，内存与排序不可忽略
$n \approx 100$	$O(n^3)$	约 100 万核心迭代，内层若很重仍要估算
$n \approx 1,000$	$O(n^2)$	约 100 万； $O(n^3)$ 通常不合适
$n \approx 100,000$	$O(n \log n)$ 、 $O(n)$	注意哈希、分配和递归栈
$n \approx 1,000,000$	优先 $O(n)$ ， $O(n \log n)$ 看限制	输入输出和内存带宽开始重要

没有通用“每秒多少次操作”的硬规则。 $n = 10^7$ 个 `int` 仅元素约 40 MB，而节点式结构、哈希桶、对齐和分配器可显著放大占用。估算状态数时还要乘每个状态的字节数。

排序与选择速查 *** / ** / *

方法	时间	稳定性 / 空间与用途
<code>std::sort</code> ***	$O(n \log n)$ 比较	不稳定；具体辅助空间依实现，常见 introsort 栈 $O(\log n)$
Merge Sort ***	$O(n \log n)$	可稳定；常用实现 $O(n)$ 缓冲，便于统计贡献
Quick Sort **	期望 $O(n \log n)$ ，最坏 $O(n^2)$	通常不稳定；随机化不能消除最坏界
Heap Sort **	$O(n \log n)$	不稳定；原地 $O(1)$ 辅助空间
Counting Sort **	$O(n+U)$	稳定版本需输出缓冲；只计频时 $O(U)$
Bucket / Radix *	依分布 / $O(d(n+B))$	桶排序最坏可退化；基数排序每轮需稳定处理
<code>nth_element</code> ***	平均 $O(n)$ （普通重载）	仅保证秩分区，不稳定、不保证两侧有序 [R1]
<code>partial_sort</code> ***	约 $O(n \log K)$	前 K 有序； K 很小时实用
大小 K 的 heap ***	$O(n \log K)$	流式处理，空间 $O(K)$

选择数据结构先问“操作” ***

操作组合	首选	不满足时
等值查找/频次	hash ***, 稠密小值域用数组	需有序则 map/set
查最小/最大并删除	heap ***	任意删除用 multiset 或惰性删除 **
下标读写、频繁顺序扫描	vector ***	双端增删用 deque; 稳定节点链接才考虑 list
静态区间和 / min	Prefix *** / Sparse Table **	有更新则 BIT/Segment Tree
动态秩 / 第 k 小	非负频次 Fenwick **	任意值插删可压缩或 order-statistics tree *
无向加边连通	DSU ***	静态 DFS; 删除转离线或更高级结构
词前缀	Trie **	单模式子串用 KMP **

DP 与图的成本展开 ***

DP 先写“状态数×每状态转移数”，例如 $dp[mask][last]$ 的状态有 $n \cdot 2^n$ 个，再枚举下一点得到 $O(n^2 \cdot 2^n)$ ，不是 $O(2^n)$ 。图先区分 V 与 E ；稠密图 E 可为 $\Theta(V^2)$ 。读完所有边本身已经需要 $\Omega(E)$ 。

复习定位：最短路决策表在第 07 章；区间数据结构比较表在第 03 章；Missing/Duplicate 决策树在第 09 章；模型识别地图在第 14 章。目录和 PDF 书签可直接跳转。

16 C++ 面试实现检查表

把正确思路写成可靠代码：边界、数值、容器语义、测试以及向面试官解释的顺序同样重要。

落笔前：输入契约 ***

确认项	要明确的内容
对象与范围	空输入是否允许？下标从 0 还是 1？值可为负、重复、极大吗？
输出与失败	求任意解还是字典序解？无解返回什么？图不连通如何表示？
可变性与顺序	能否排序/修改输入？是否需要保留原下标？查询能否重排？
复杂度与数值	n、Q、值域、内存上限；最大中间值；递归深度；结果是否取模？
C++ 版本	默认标准库可用；确认 C++17/20 与编译器扩展许可。

实现过程中：高频错误 ***

检查点	正确做法 / 典型陷阱
迭代器失效	vector 扩容使全部迭代器/引用失效；erase 使擦除点及之后失效。unordered rehash 使迭代器失效，但通常不使元素引用/指针失效；删除该元素当然失效。
有序容器	map/set 删除仅使被删元素的迭代器失效；需要 O(log n) 定位用成员 lower_bound。multiset 删除一个应 erase(iterator)。
reserve vs resize	reserve 只分配容量，不创建可索引元素；resize 才改变 size。不能 reserve(n) 后直接 a[i] 赋值。
比较器	严格弱序，不能用 <=；按多个键时明确相等规则。priority_queue 的比较器方向要手动验证。
整数与 INF	先扩宽再乘；先判可达再相加；可用 INF=max/4 但仍须确保合法有限结果小于它。
unsigned 与位移	空容器 size()-1 会下溢；位移数必须小于位宽；不要依赖有符号溢出。
状态初始化	未访问与合法结果 0 分开；最小化用 +INF，最大化不可达用 -INF；dp[0] 的含义要说明。
引用与副本	auto [x,y] 可能复制；读大对象用 const auto&；不要保存可能失效的引用。
特殊容器	vector<bool> 是代理位表示，不能当普通 bool 引用数组；string_view/span 不拥有底层数据。
STL 接口	priority_queue::pop() 不返回值，先 top 再 pop；queue 同理。nth_element 的 nth 与 k-1 分清。

完成后：最小但有效的测试集 ***

类别	示例 / 观察点
规模边界	空、1 个、2 个、最大深度链状树；空输入若不在契约内，也要清楚说明。
值边界	全相同、严格递增/递减、全负、零、数值上界；重复值检验严格性。
答案边界	答案在第一/最后位置、无解、全部都合法、只一个合法；K=0/1/n。
结构边界	孤立点、多分量、自环、平行边、负边、环；只测试输入契约允许的情况。
随机对拍	小规模随机输入，与暴力或更简单解法比较；失败样例固定种子保存。
运行诊断	可用时启用编译警告、AddressSanitizer/UBSan；性能数据在优化构建下测量。

Live Coding 表达顺序 ***

1. 复述目标与约束。2. 给朴素解及瓶颈。3. 用第 02 章的思维框架说明转换、排除、合并或复用的依据。4. 定义状态与不变量。5. 实现主体与边界。6. 用普通及极端例子走查。7. 汇总时间、空间和可改进点。

Senior / 低延迟追问 **

在相同渐进复杂度下，解释容器选择对连续扫描、指针追踪、内存分配和迭代器稳定性的影响。若要求低延迟，区分均值与高分位延迟，并指出可能触发 rehash、扩容、递归深度或堆积惰性删除的路径。先测量，再改布局。并发版本必须补齐线程数量、所有权、内存顺序与回收约束。

复习完成标准 *** / ** / *

优先级	达标检查
*** 基础 / 必须掌握	能独立识别、给出不变量、实现模板，说明复杂度，并举出失效反例。
** 需要掌握	能说明适用条件与核心推导，读懂并改写主体，知道如何验证。
* 可以了解 / 高阶扩展	能说出它解决什么、替代谁、需要哪些额外条件；按岗位决定是否练实现。

核验资料与进一步阅读

正文依据原对话重新编排与补充；以下为少量细节的核验与深入阅读入口，并非逐段摘录。算法表达与代码为本手册统一整理。

- [R1] C++ 工作草案：nth_element 的契约与复杂度
<https://eel.is/c++draft/alg.nth.element>
- [R2] CP-Algorithms: 稀疏图 Dijkstra 与过期堆状态
https://cp-algorithms.com/graph/dijkstra_sparse.html
- [R3] CP-Algorithms: DSU、距离与奇偶扩展
https://cp-algorithms.com/data_structures/disjoint_set_union.html
- [R4] CP-Algorithms: Prefix Function / KMP
<https://cp-algorithms.com/string/prefix-function.html>
- [R5] C++ 工作草案：bit_ceil 等二次幂操作
<https://eel.is/c++draft/bit.pow.two>

模板使用说明：先阅读每段代码上方的输入契约。所有代码块作为同一 C++17 测试单元做编译与代表性/随机小样本核对；这覆盖展示模板，不等同于对全部文字扩展算法提供实现或形式化验证。